

A PROJECT REPORT ON

**Detecting oil spills at marine environment using
Automatic Identification System (AIS) and satellite
datasets**

**SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE**

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Omkar Shinde
Amey Wadgaonkar
Asmita Mhetre

Exam No: B400050595
Exam No: B400050635
Exam No: B400050505



**DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043**

**SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025**



CERTIFICATE

This is to certify that the project report entitled

**Detecting oil spills at marine environment using
Automatic Identification System (AIS) and satellite datasets**

Submitted by

Omkar Shinde
Amey Wadgaonkar
Asmita Mhetre

Exam No: B400050595
Exam No: B400050635
Exam No: B400050505

are bonafide students of this institute and the work has been carried out by them under the supervision of **Dr. G. P. Potdar** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Dr. G. P. Potdar
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place: Pune
Date:

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**Detecting oil spills at marine environment using Automatic Identification System (AIS) and satellite datasets**’.*

*We would like to take this opportunity to thank **Dr. G. P. Potdar** for giving us all the help and guidance we needed. We are really grateful to him for his kind support. His valuable suggestions were very helpful.*

*We are also grateful to **Dr. G. V. Kale**, Head of Computer Engineering Department, PICT for her indispensable support and suggestions throughout the project. We also express our gratitude to **Dr. A. R. Deshpande**, BE project coordinator, for her timely guidance and support.*

*We would also like to thank our respected **Dr. S. T. Gandhe**, Principal, PICT, for his constant encouragement, continued support, and invaluable feedback which motivated us throughout the duration of the project.*

*In the end our special thanks to our honorable **Dr. P. T. Kulkarni**, Director, PICT, for his continued support and valuable inputs during the ideation and implementation phase of the project. We also thank our laboratory assistants for their valuable help in setting up the workstations. We are deeply thankful for the platform and resources that PICT offers us, allowing us to undertake this valuable project.*

Omkar Shinde
Amey Wadgaonkar
Asmita Mhetre
(B.E. Computer Engg.)

ABSTRACT

Oil spills have devastating effects on marine ecosystems, threatening marine life and coastal environments. They arise from sources like tanker accidents, illegal discharges, and natural seabed seepage. Synthetic Aperture Radar (SAR) is crucial for oil spill detection, offering all-weather monitoring capabilities. However, distinguishing spills from similar phenomena, like biogenic slicks and ship wakes, is challenging. Detection techniques include thresholding, edge detection, texture analysis, and polarimetric decomposition. Recently, machine learning (ML) and deep learning (DL) methods, especially Convolutional Neural Networks (CNNs), have improved detection accuracy, automating feature extraction from SAR data. Integrating SAR with other data sources, like AIS, enhances detection and provides a fuller view of spills.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition and Objectives	2
1.4	Project Scope & Limitations	3
1.5	Methodologies of Problem solving	4
2	Literature Survey	6
3	Software Requirements Specification	9
3.1	Assumptions and Dependencies	10
3.2	Functional Requirements	10
3.2.1	System Feature 1: Core Functionality	10
3.2.2	System Feature 2: User Experience and Interface Design Requirements	10
3.2.3	System Feature 3: Data Privacy	10
3.3	External Interface Requirements	11
3.3.1	User Interfaces	11
3.3.2	Hardware Interfaces	12
3.3.3	Software Interfaces	12
3.4	Nonfunctional Requirements	12
3.4.1	Performance Requirements	12
3.4.2	Safety Requirements & Security Requirements	13
3.5	System Requirements	13
3.5.1	Database Requirements	13
3.5.2	Software Requirements	13
3.5.3	Hardware Requirements	14
3.6	Analysis Models: Agile Methodology to be applied	14
4	System Design	16
4.1	System Architecture	17
4.2	Mathematical Model	20
4.3	State Machine Diagram	21
4.4	Use Case Diagram	22
4.5	Entity Relationship Diagram	23
4.6	Class Diagram	24
4.7	Sequence Diagram	25
4.8	Deployment Diagram	26

5	Project Plan	27
5.1	Project Estimate	28
5.1.1	Reconciled Estimates	28
5.1.2	Project Resources	28
5.2	Risk Management	29
5.2.1	Risk Identification	30
5.2.2	Risk Analysis	30
5.3	Project Schedule	32
5.3.1	Project Task Set	32
5.3.2	Timeline Chart	33
5.4	Team Organization	34
5.4.1	Team structure	34
5.4.2	Management reporting and communication	34
6	Project Implementation	35
6.1	Overview of Project Modules	36
6.2	Tools and Technologies Used	36
6.3	Algorithm Details	37
6.3.1	Algorithm 1: AIS Data Collection and Preprocessing	37
6.3.2	Algorithm 2: Anomaly Detection in Vessel Movement	37
6.3.3	Algorithm 3: Satellite Image Retrieval from GEE	38
6.3.4	Algorithm 4: Image Preprocessing for CNN Input	38
6.3.5	Algorithm 5: CNN-Based Oil Spill Classification	38
6.3.6	Algorithm 6: Alert Generation and Logging	38
6.3.7	Algorithm 7: Dashboard Visualization Algorithm	38
6.3.8	Algorithm 8: Historical Event Review and Feedback Loop	39
7	Software Testing	40
7.1	Types of Testing	41
7.2	Test cases & Test Results	41
8	Results	43
8.1	Outcomes	44
8.2	Screen Shots	44
9	Conclusions	52
9.1	Conclusion	53
9.2	Future Work	53
9.3	Applications	54
10	Appendix	55
10.1	Appendix A: Problem Statement Feasibility Assessment	56
10.2	Appendix B: Paper Publication Details	57
10.3	Appendix C: Plagiarism Report	58
11	References	59

List of Figures

4.1	System Architecture	18
4.2	State Machine Diagram	21
4.3	Use Case Diagram	22
4.4	Entity Relationship Diagram	23
4.5	Class Diagram	24
4.6	Sequence Diagram	25
4.7	Deployment Diagram	26
5.1	Summarized TimeLine Chart	33
8.1	Home Page	44
8.2	Main Page	45
8.3	Page rendering Top 10 Most Dangerous Ships	46
8.4	Contact us Page	47
8.5	Connecting to the server to collect live information about all nearby ships using MQTT	48
8.6	Detecting oil spill intensity and setting threshold levels to assess security risks at sea	49
8.7	Downloading a .tif image highlighting the detected oil spill area	49
8.8	Dataset Used	50
8.9	Image showing the area where an oil spill has been detected	51
10.1	Plagiarism Report Snapshots for Various Project Sections	58

CHAPTER 1

INTRODUCTION

1.1 Overview

Oil spills represent a significant and devastating threat to marine ecosystems, coastal areas, and related industries. Therefore, achieving early detection and continuous monitoring is crucial for mitigating their adverse impacts. Traditional methods heavily rely on satellite-based Synthetic Aperture Radar (SAR) data due to its valuable capability to operate in various weather conditions and during darkness. However, a primary challenge in using SAR data is the difficulty in distinguishing actual oil spills, which often appear as dark patches, from other similar phenomena known as "look-alikes," such as biogenic slicks, low wind areas, or natural seepage. To address this limitation and enhance detection accuracy, this project explores the integration of SAR data with Automatic Identification System (AIS) information. AIS transponders on vessels provide real-time data regarding ship locations and movements, which can be instrumental in pinpointing the potential source of a spill, predicting oil drift, and refining the interpretation of SAR images. The project uses machine learning and deep learning methods, including Convolutional Neural Networks (CNNs). It involves a system designed to collect AIS and satellite data, process and analyze it through modules such as anomaly detection, store the information in a database, and generate outputs like alerts and visualizations through a user interface. This integrated approach aims to build an effective automated system for oil spill detection by leveraging the strengths of both satellite remote sensing and maritime traffic data analysis.

1.2 Motivation

This project addresses the critical challenge of accurately detecting marine oil spills by leveraging both satellite and maritime data. Traditional satellite-based surveillance using Synthetic Aperture Radar (SAR) is effective in capturing sea surface imagery under various conditions; however, its reliability is hindered by the difficulty of distinguishing real oil spills from similar dark features—commonly known as "look-alikes"—such as biogenic slicks, calm water zones, ice, or natural seeps. To improve detection accuracy and reduce false positives, this system integrates SAR data with Automatic Identification System (AIS) data, which provides real-time information on vessel identity and movements. This fusion enables better contextual analysis, such as linking dark patches to nearby ships or predicting oil drift patterns. Advanced machine learning and deep learning techniques, particularly Convolutional Neural Networks (CNNs), are applied to analyze and classify the data more effectively. The project aims to develop a robust, data-driven detection system that enhances situational awareness and supports timely environmental response actions.

1.3 Problem Definition and Objectives

Problem Definition:-

Oil spills severely impact marine ecosystems and require timely detection for effective response. The proposed work proposes an integrated approach using Automatic Identification System (AIS) data and satellite imagery (e.g., SAR and optical) to automatically detect oil spills by analyzing vessel movement patterns and surface anomalies, enabling faster and more accurate environmental monitoring.

Objectives:-

1. Design and implement a comprehensive system architecture capable of collecting, processing, and integrating both satellite Synthetic Aperture Radar (SAR) imagery and Automatic Identification System (AIS) data for marine oil spill detection.
2. Develop and apply machine learning and deep learning models, such as Convolutional Neural Networks (CNNs) or U-Net architectures, to analyze SAR images for the identification and semantic segmentation of potential oil spill areas, as well as other marine targets like ships and look-alikes.
3. Leverage processed AIS vessel tracking data to provide contextual information for SAR image analysis, including the presence and behavior of ships in proximity to detected dark features, to assist in distinguishing actual oil spills from "look-alikes" and potentially identifying pollution sources.
4. Implement necessary data processing techniques, including image preprocessing for SAR, parsing of AIS data, feature extraction, and strategies to handle issues such as class imbalance, to enhance the accuracy and reliability of the detection models.
5. Evaluate the performance of the developed integrated oil spill detection system and its individual components using appropriate evaluation metrics, such as classification accuracy, precision, recall, F1-score, Dice coefficient, Intersection over Union (IoU), or ROC analysis, based on available datasets.

1.4 Project Scope & Limitations

Project Scope:-

This project focuses on developing an integrated system for the detection of marine oil spills by combining Synthetic Aperture Radar (SAR) satellite imagery with Automatic Identification System (AIS) data. The core objective is to improve detection accuracy and reduce false positives by addressing the challenge of differentiating actual oil slicks from look-alike features such as low wind zones or natural seepage. The system employs machine learning and deep learning models, including Convolutional Neural Networks (CNNs) for classification and U-Net for segmentation of SAR imagery. AIS data is incorporated to associate vessel movements with detected slicks, aiding in the identification of potential polluters. The project involves designing data preprocessing pipelines, implementing model training and evaluation modules, and integrating outputs into a functional system. Performance is assessed using standard metrics such as accuracy, precision, recall, Dice coefficient, and Intersection over Union (IoU).

Limitations:-

Synthetic Aperture Radar (SAR) sensors vary in frequency, polarization, and acquisition mode, all of which affect their effectiveness in oil spill detection. For instance, modes like ScanSAR Wide provide extensive coverage but at lower spatial resolutions (e.g., 100 meters), limiting their ability to detect smaller or thinner slicks. Detection performance is also influenced by polarization; HH polarization becomes less reliable at large incidence angles, particularly under low wind conditions. Environmental factors such as high winds, heavy rainfall, or extremely calm seas can disrupt radar backscatter, mak-

ing oil spills indistinguishable from the background. SAR-based detection is typically effective within a moderate wind speed range of approximately 2–10 m/s; outside this range, both radar and optical-based systems (e.g., UAV imagery) tend to underperform. Deep learning models may also fail when trained on insufficient or imbalanced datasets, especially in multi-class classification scenarios where oil features are intermixed with look-alike phenomena like biogenic films, low wind zones, and natural seepage. Additionally, the current system performs optimally on datasets up to several thousand labeled SAR image patches; scalability beyond this requires extensive computational resources and advanced data augmentation strategies to maintain accuracy.

1.5 Methodologies of Problem solving

The project follows a structured methodology beginning with the initial setup and environment preparation, which involves configuring programming environments, data processing libraries, and version control systems like Git to ensure a smooth development process. The next major phase is comprehensive data handling. This includes a data survey and collection process that focuses on obtaining SAR satellite imagery and AIS data, evaluating their quality, accessibility, and relevance. Challenges such as the high cost of satellite data, limited availability of large labeled datasets, and inconsistencies in historical AIS data are also addressed. Data parsing and preprocessing involve converting raw AIS data into usable formats and applying techniques like land masking, speckle reduction, and histogram equalization to SAR images. Additional steps such as data augmentation and balancing are used to improve the quality and usability of the dataset.

In the model development and training phase, suitable machine learning and deep learning models—typically CNN-based architectures like U-Net or DeepLabv3+—are selected and implemented. This includes defining activation functions, segmentation layers, and exploring both automatic and explicit feature extraction techniques based on texture, shape, and intensity. Model training requires hyperparameter tuning with validation datasets and may benefit from ensemble modeling to improve performance. System integration involves merging SAR and AIS data for multisource analysis, which enhances the ability to detect and verify oil spills. Software systems are developed with database connectivity, API implementation, and communication interfaces to support broader usage, although technical integration challenges are acknowledged.

The next step involves prototyping, testing, and evaluation. Preliminary tests are carried out to refine the system, followed by formal evaluation using metrics like accuracy, precision, recall, F1-score, Dice coefficient, and IoU. Advanced validation methods, such as leave-one-batch-out, are considered to ensure robust performance assessments. Key challenges addressed throughout the methodology include differentiating real oil spills from look-alikes using contextual and visual features, managing data imbalance to improve classification, and dealing with environmental factors that may affect detection accuracy.

The oil spill detection system is based on visual data in the form of black and white (grayscale) satellite images of water bodies. The primary datatype used is single-channel grayscale imagery, typically in .jpg, .png, or .tiff formats. These images are represented as two-dimensional matrices in MATLAB, where each pixel holds an intensity value rang-

ing from 0 (black) to 255 (white). This datatype is well-suited for detecting oil spills, as oil-affected regions generally appear as darker patches compared to the surrounding water surface. The simplicity of grayscale data reduces computational complexity and makes it easier to isolate regions of interest based on intensity variations.

The dataset used for the implementation was primarily sourced from the Smart India Hackathon (SIH), which provided a collection of satellite images specific to the oil spill detection problem statement. Additionally, further data was gathered from other participants who had chosen the same problem during previous SIH events. Altogether, the dataset consists of around 500–600 images, ensuring diversity in geography, lighting, and environmental conditions. During the system pipeline, different images are used for different stages: preprocessing involves the entire dataset for normalization and noise removal; segmentation is tested on images with clear contrast between oil and water; and feature extraction is validated on samples containing annotated or visually confirmed oil spill regions. The dataset used in this project comes from three key sources. First, a primary batch of static satellite images was obtained through the Smart India Hackathon (SIH), as part of the official problem statement. Second, additional images were collected from peer participants who had worked on the same problem in previous SIH events, increasing data diversity and case coverage. Third, and most critically, the system integrates a real-time dataset from a Fresh Water Monitoring API, which delivers up-to-date environmental parameters in JSON format. This data includes readings such as turbidity, pH, chemical content, and possibly surface reflectance indicators—each of which may correlate with the presence of oil contamination. These values are parsed and matched with location metadata to support live detection decisions or to flag potential spill zones even before visual confirmation.

To ensure quality and consistency, data cleaning techniques were applied at every stage. For static image data, each image was resized to a uniform dimension (e.g., 512×512 pixels) and filtered using median filters and Gaussian blur to remove noise while preserving essential edge details. Histogram equalization was used for contrast enhancement. The JSON data from the API is first parsed and validated for completeness, then filtered to remove outliers using statistical thresholds (e.g., z-score filtering). Irrelevant or missing values are excluded to avoid false positives. Cleaned data is stored in a structured local directory with proper versioning. Image data is stored in MATLAB .mat format, while real-time JSON entries are logged into structured .csv or .json files along with timestamps for traceability and audit purposes. This data management setup allows for scalable processing and easy integration with future machine learning modules.

CHAPTER 2

LITERATURE SURVEY

Oil spill detection in marine environments has evolved significantly with advancements in satellite remote sensing and artificial intelligence. A key component of operational ocean monitoring systems—such as the EU-funded integrated Mediterranean Sea Observing System—is the use of SAR (Synthetic Aperture Radar) imagery. These systems combine raw satellite data with labeled “oil look-alike” features to evaluate detection algorithms, offering broad regional coverage while highlighting the growing role of AI in ocean surveillance workflows [1].

One notable approach involves the use of neural networks alongside classic image processing methods. Roussio et al. (2022) proposed a sequence of methods for accurate oil spill detection, emphasizing the importance of preprocessing tools, CNN models, and specific datasets tailored for oil spill recognition challenges. Their methodology includes SAR image filtering to highlight the unique physical characteristics of oil spills, followed by the application of leading CNN architectures and ensemble modeling to reduce generalization errors [1].

The SAR processing chain typically begins with image acquisition, followed by georeferencing, land masking, speckle filtering, and threshold-based segmentation. For instance, studies utilizing RADARSAT imagery demonstrate how slick candidates can be visualized with probabilistic color-coded outputs, providing a foundation for modeling oil spill trajectories [2].

In a similar vein, Huang et al. (2022) developed a novel deep learning method employing the Faster Region-based Convolutional Neural Network (Faster R-CNN) model for automatic detection of marine oil spills from satellite SAR imagery. Their approach addresses limitations in existing methods by offering a fast and effective detection mechanism capable of handling imbalanced datasets and uncertainties in feature selection [2].

Recognizing the limitations of SAR alone—especially its vulnerability to false positives caused by look-alikes such as biogenic films or calm water—researchers have explored multi-source data fusion. Integrating SAR with optical sensors (like MODIS), bathymetry, and AIS (Automatic Identification System) data enhances the ability to verify oil spills and identify potential sources. AIS vessel tracks, when overlaid with SAR detections, have proven effective in flagging likely polluters, as shown in case studies using Radarsat-2 imagery [4].

Chaudhary and Kumar (2020) focused on the detection of marine oil slicks using both spaceborne and airborne SAR data. Their study reviewed the potential of quad-pol and hybrid-pol datasets, utilizing UAVSAR and RISAT-1 data acquired during oil spill experiments. The research highlights the effectiveness of polarimetric decomposition techniques and supervised classification methods in distinguishing oil spills from look-alike features [3].

AIS data has also gained traction in anomaly detection applications, where deep learning models treat AIS messages like network packets. By extracting temporal and behavioral features, such systems can detect deviations from normal vessel patterns, complementing SAR-based detection with real-time traffic analysis [3,13].

The integration of Automatic Identification System (AIS) data has also been explored to enhance maritime navigation and oil spill detection. Tu et al. (2016) conducted a comprehensive survey on exploiting AIS data for intelligent maritime navigation, discussing its applications in traffic anomaly detection, route estimation, collision prediction, and path planning. The study underscores the value of AIS data in complementing remote sensing technologies for improved maritime safety and efficiency [4].

Furthermore, Liu et al. (2020) proposed a deep-learning framework for large-scale detection and categorization of oil spills from SAR images. Their model, designed for image segmentation, was trained on an extensive dataset, achieving state-of-the-art performance comparable to human operators. The study also introduced a novel classification task within the context of oil spill detection in SAR imagery [5].

On the machine learning front, several surveys categorize traditional and deep learning methods for SAR-based oil spill detection. Support Vector Machines (SVM), Random Forests, and more recently Convolutional Neural Networks (CNNs) and U-Nets have shown promise, especially when large, annotated datasets are available. The importance of dataset quality, standardized benchmarking, and generalizable architectures is a recurring theme in the literature [5,8,9].

Advanced approaches, such as genetic algorithms for threshold optimization and morphological postprocessing, offer adaptability to varying sea states and can outperform traditional segmentation techniques in certain scenarios [6]. Environmental metadata—like wind vectors, sea currents, and wave direction—are increasingly incorporated to contextualize detections and refine spill trajectory forecasts [7,14].

Meta-analyses covering decades of SAR-based spill research reveal key trends, such as the shift toward deep learning post-2015 following open-access policies like ESA's Sentinel-1 program. These studies quantify sensor usage, evaluate polarization settings, and assess the methodological evolution from rule-based systems to AI-driven frameworks [10,11].

Polarimetric analysis further expands detection capabilities. Techniques like Van Zyl and Yamaguchi decomposition, applied to UAVSAR and RISAT-1 data, have achieved detection accuracies up to 86% using classifiers like Wishart and SVM. However, differences in data sources and preprocessing pipelines pose challenges for reproducibility and cross-study comparisons [12].

Finally, integrated platforms such as SeaScope demonstrate the operational potential of combining Sentinel-1 SAR, AIS, and metocean data. By classifying slicks based on source—such as vessel discharge versus offshore production—these systems represent a leap forward in real-time maritime surveillance and response [15].

CHAPTER 3

**SOFTWARE REQUIREMENTS
SPECIFICATION**

3.1 Assumptions and Dependencies

Assumptions:-

1. The system assumes that users will have access to high-quality Synthetic Aperture Radar (SAR) and Automatic Identification System (AIS) data for effective oil spill detection.
2. It is assumed that users possess the necessary technical skills to operate the software and interpret the results generated by the detection algorithms.
3. The effectiveness of machine learning models relies on the availability of sufficient labeled training data for supervised learning approaches.

Dependencies:-

1. The software depends on external data sources, which include: SAR Data and AIS Data.
2. The software may also depend on machine learning frameworks and libraries (e.g., TensorFlow, PyTorch) for implementing deep learning algorithms.

3.2 Functional Requirements

3.2.1 System Feature 1: Core Functionality

Description:-

This subsection details the primary operations the system is required to carry out.

Requirements:-

1. The system should be capable of processing and analyzing SAR and AIS data to detect oil spills accurately.
2. This includes implementing machine learning algorithms, particularly deep learning techniques like Convolutional Neural Networks (CNNs), to enhance detection capabilities.
3. Integrating data from multiple sources (SAR, AIS, meteorological data) to improve detection accuracy and provide a comprehensive view of potential spill incidents.

3.2.2 System Feature 2: User Experience and Interface Design Requirements

Description:-

This subsection covers the requirements related to how users will interact with the system and the design of the user interface.

Requirements:-

1. Visualize detected oil spills on maps with clear indicators and overlays.
2. Access historical data and analysis tools for better decision-making.
3. Receive notifications and alerts regarding new detections or changes.

3.2.3 System Feature 3: Data Privacy

Description:-

This subsection outlines the requirements to ensure the privacy and security of the data

handled by the system.

Requirements:-

1. The system must comply with relevant data protection regulations.
2. This compliance is to ensure that AIS data and other sensitive information are handled securely.
3. User data is anonymized where possible to protect individual privacy

3.3 External Interface Requirements

3.3.1 User Interfaces

Feature	Description
Navigation and Functionality Access	Enables users to navigate the system intuitively and access various functionalities efficiently.
Visualization of Detections	Provides map-based visualization of detected oil spills with clear indicators and overlays.
Access to Historical Data & Tools	Allows users to retrieve historical data and utilize analysis tools for informed decision-making.
Notifications and Alerts	Delivers timely notifications and alerts regarding new detections or significant changes.
Information Clarity	Ensures that all information is presented in a clear and concise manner for user comprehension.
Customization	Offers users the ability to customize views and reports to suit specific requirements.
Accessibility	Guarantees accessibility across various devices, including desktops and mobile platforms, with a user-friendly interface.

Table 3.1: User Interfaces

3.3.2 Hardware Interfaces

Hardware Component	Description
Computers	Compatible desktops or laptops for running the application.
Network Connection	Reliable internet connection for server communication and data reporting.

Table 3.2: Hardware Interfaces

3.3.3 Software Interfaces

Software Interface	Description
Data Processing Algorithms	Interfaces for integrating machine learning and deep learning algorithms for oil spill detection, such as Convolutional Neural Networks (CNNs).
Database Management Systems	Connections to scalable databases for storing and retrieving historical oil spill data and vessel movement records, including AIS, detection, temporal, and alert data.
APIs	Application Programming Interfaces enabling third-party systems (e.g., Maritime Authorities, Emergency Response) to access detection data and analytics.

Table 3.3: Software Interfaces

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

1. Response Time: The system should provide real-time detection and alerts for oil spills with a maximum response time specified as 'X'.
2. Throughput: The system must be able to process a minimum of 'X' data points per second to effectively manage the large amounts of incoming satellite and AIS data.
3. Scalability: The system needs to be designed for horizontal scaling to handle growing data volumes and increasing user demand without any performance degradation.

3.4.2 Safety Requirements & Security Requirements

1. **Data Protection:** The system must implement encryption for data at rest and in transit to protect sensitive information from unauthorized access.
2. **User Authentication:** The system should require strong user authentication mechanisms (e.g., multi-factor authentication) to ensure that only authorized personnel can access the system.
3. **Access Control:** Role-based access control (RBAC) should be implemented to restrict user access to sensitive data and functionalities based on their roles.
4. **Incident Response:** The system should have a defined incident response plan to address potential security breaches or data leaks, including notification procedures for affected parties.

3.5 System Requirements

3.5.1 Database Requirements

1. **Data Storage:** We employed a NoSQL database (MongoDB) to store large volumes of historical and real-time data related to AIS signals, vessel anomalies, satellite imagery metadata, and oil spill detection zones. The database structure was designed to handle both structured and semi-structured maritime data efficiently.
2. **Data Retrieval:** Data retrieval was performed through RESTful APIs integrated with the backend, allowing efficient geospatial and temporal querying. This enabled real-time analysis of vessel behavior, correlation with satellite data, and dynamic updates on the monitoring dashboard.
3. **Data Backup and Recovery:** We implemented automated backup procedures and a disaster recovery strategy to ensure data availability and integrity. This helped safeguard critical AIS and satellite data against potential system failures or outages.
4. **Scalability:** The database system was designed to be horizontally scalable, allowing it to accommodate increasing data volumes over time as the number of monitored vessels and geographic regions expanded.

3.5.2 Software Requirements

1. **Framework:** PyTorch for model development and testing; Flask or Django for web deployment.
2. **Languages:** Python for core development; C/C++ for legacy components and integration with MS Visual C++.
3. **Libraries:** TensorFlow and PyTorch for deep learning; Pandas and Apache Spark

for data processing; libais for AIS data parsing; Segmentation Models Repository for CNN implementation.

3.5.3 Hardware Requirements

1. **Server Specifications:** The system requires servers with specific specifications, including a multi-core processor, sufficient memory (e.g., 16GB RAM), and storage (e.g., SSDs for fast data access).
2. **Network Infrastructure:** High-speed internet connectivity is needed for accessing satellite data and AIS information.
3. **Backup Systems:** The hardware setup should include backup systems to ensure data redundancy and recovery capabilities.
4. **Computing Hardware for Processing:** Requirements for servers or cloud infrastructure are necessary to process and analyze large datasets efficiently. Note that one study also mentions using a workstation with an NVIDIA GeForce RTX 3090 GPU for oil spill detection, achieving a computer runtime of less than 0.05 seconds per full SAR image, but this level of specific detail is not in the general hardware requirements section.

3.6 Analysis Models: Agile Methodology to be applied

We followed an Agile methodology during the development of our analysis models, breaking the work into iterative sprints to ensure modular progress, flexibility, and continuous feedback integration. The process was structured as follows:

1. **Iteration Planning:** At the start of each sprint, we identified and prioritized specific tasks such as refining the CNN model for detecting oil spill patterns or implementing the initial version of the Trajectory Analysis module. Prioritization was based on project goals and feedback gathered during previous reviews.
2. **Environment and Tool Setup:** We set up the required development environments, including Python, TensorFlow, PyTorch, and relevant geospatial libraries, tailored for the machine learning and data processing tasks of each iteration.
3. **Data Preparation:** Each sprint involved preparing data specific to the models being developed, including acquiring AIS data and satellite imagery. We performed preprocessing such as parsing, noise reduction, data augmentation, and feature extraction to ensure the datasets were suitable for training and evaluation.
4. **Model Design and Implementation:** We designed and implemented models such as CNNs for oil spill classification and trajectory-based modules for vessel anomaly detection. This included writing modular code and integrating techniques using the selected tools and libraries.

5. Testing and Evaluation: We evaluated our models using standard performance metrics like accuracy, precision, recall, and Dice score. These evaluations helped us establish baselines and identify areas for further refinement in subsequent sprints.

6. Integration and Refinement: After validation, we integrated the developed components into the main system. Based on feedback and performance evaluations, we refined the algorithms iteratively to improve the system's overall accuracy and responsiveness.

7. Documentation and Review: Comprehensive documentation was maintained for each sprint, including methodology, challenges encountered, and implementation details. At the end of each iteration, we conducted review meetings to assess progress and define the scope for the next sprint.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The System Architecture Diagram involves various components:

I. User Machines (1, 2, ..., n):

Each user machine generates data related to system usage, performance, and activity. This data is sent to a central system for tracking and management purposes.

II. Store Machine Data:

The collected data from user machines is centralized and stored systematically. It includes details such as application usage, system health, and performance metrics.

III. Database:

A centralized database stores machine data for easy access and management. It supports future retrieval and analysis by organizing data from multiple machines.

IV. Lab Manager / Administrator:

The lab manager monitors machine data to ensure optimal system performance. They use the system to analyze user behavior and machine health reports.

V. Extract / Process / Transform / Load (ETL):

Data is extracted from the database and processed for analysis. It undergoes transformation to prepare for reporting and visualization tasks.

VI. Reporting, Visualization, and System Analysis:

Processed data is used to generate reports for system performance and user activities. Visualization and system analysis help in understanding patterns and optimizing the lab environment.

System Architecture Flow (Based on the Diagram):-

I. Input Sources

The architecture begins with multiple data input streams which are integrated for further processing:

- AIS Receivers – Capture Automatic Identification System signals from vessels.
- Satellite Data – Provide wide-area surveillance and weather information.
- Weather Data – Supplies real-time meteorological conditions affecting maritime operations.
- Ship Database – Offers static data about vessel specifications and historical records.

All inputs flow into the Data Integration Layer, which aggregates these diverse datasets into a unified format.

II. Processing Core

This core stage is responsible for preparing the integrated data for in-depth analysis:

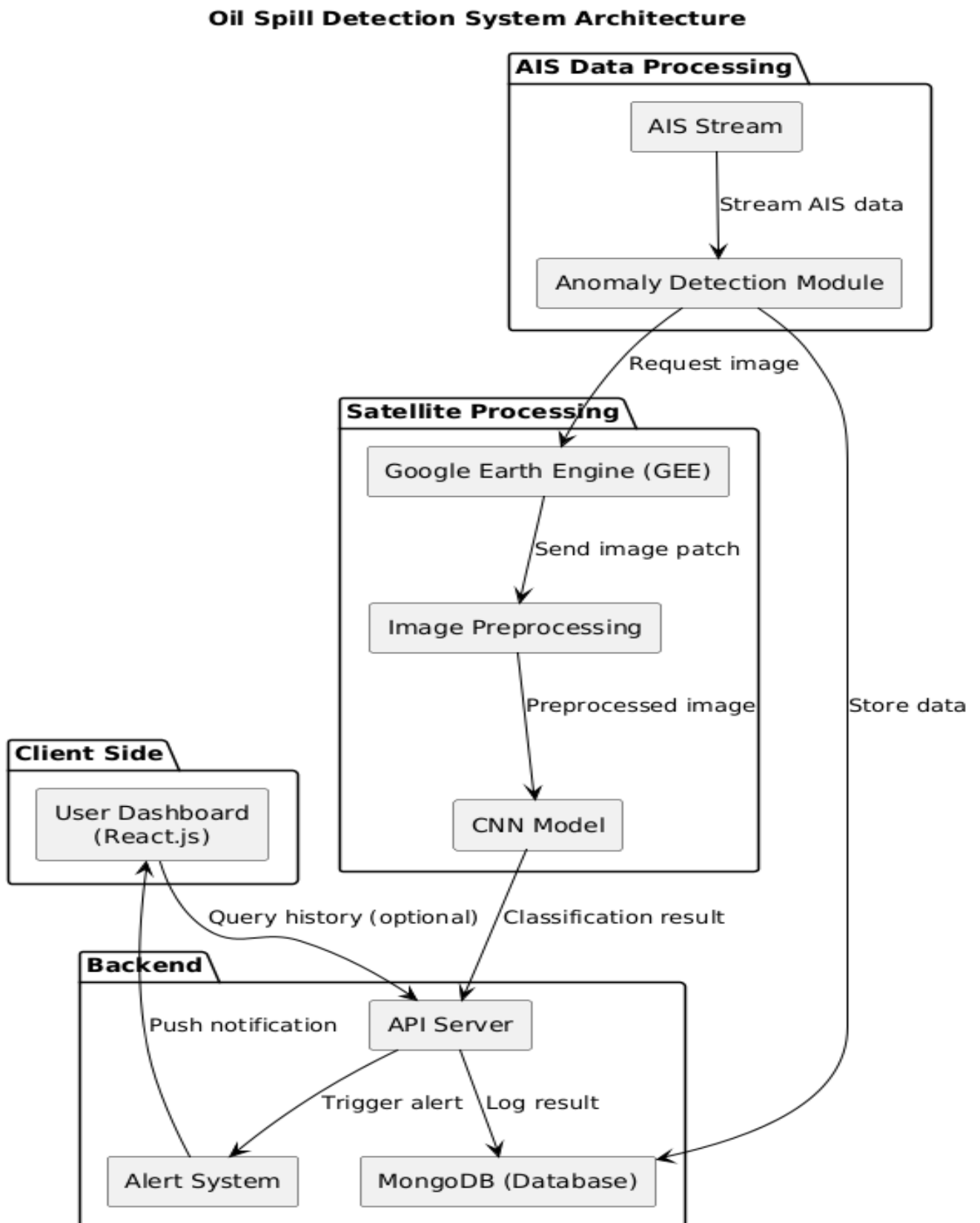


Figure 4.1: System Architecture

- Data Preprocessing – Cleans, validates, and formats raw data.

- Feature Extraction – Identifies key attributes and patterns from the cleaned data.
- Analysis Engine – Serves as the computational heart, orchestrating all analytical operations.

III. Analysis Modules Specialized modules execute distinct types of analysis:

- Trajectory Analysis – Studies vessel paths for navigation tracking.
- Pattern Recognition – Detects routine or recurring vessel behaviors.
- Risk Assessment – Evaluates safety hazards based on dynamic and historical data.
- Anomaly Detection – Flags irregular or suspicious vessel activities.

IV. Output Layer This layer transforms analysis outcomes into actionable intelligence:

- Decision Support System – Aggregates analysis results to aid strategic decisions.
- Alert System – Triggers real-time alerts for critical events.
- Reporting Module – Compiles structured summaries for stakeholders.
- Visualization Interface – Offers graphical insights into maritime data.

V. External Systems The final outputs are directed toward operational stakeholders:

- Maritime Authorities
- Emergency Response Teams
- Coast Guard

4.2 Mathematical Model

Let the system be represented as a tuple:

$$S = \{I, P, O, F, D, A, R\}$$

Where:

- I = Input Set
 $I = \{\text{SAR_Images}, \text{AIS_Data}, \text{Environmental_Parameters}, \text{Historical_Spill_Records}\}$
- P = Set of Processes
 $P = \{\text{Preprocessing}, \text{Feature_Extraction}, \text{Classification}, \text{Fusion}, \text{Validation}\}$
- O = Output Set
 $O = \{\text{Detected_Oil_Spills}, \text{Spill_Maps}, \text{Confidence_Scores}, \text{Alert_Reports}\}$
- F = Functions
 $F = \{f_1, f_2, f_3, f_4, f_5\}$ where:
 - f_1 : Ingest and preprocess SAR and AIS data
 - f_2 : Extract features from SAR images
 - f_3 : Classify detected objects using ML/DL
 - f_4 : Correlate with AIS data for anomaly detection
 - f_5 : Generate and visualize detection outputs
- D = Database
 $D = \{\text{GeoDB}, \text{SAR_Archive}, \text{AIS_Log}, \text{Spill_TrainingSet}\}$
- A = Actors
 $A = \{\text{Satellites}, \text{ML_Engine}, \text{Analysts}, \text{Maritime_Authorities}\}$
- R = Set of Rules/Conditions
 $R = \{\text{if SAR anomaly detected \& AIS mismatch} \rightarrow \text{flag as potential spill},$
 $\quad \text{if confidence} \geq \theta \rightarrow \text{auto-report to authority},$
 $\quad \text{if detected area} > \text{threshold} \rightarrow \text{trigger emergency response}\}$

4.3 State Machine Diagram

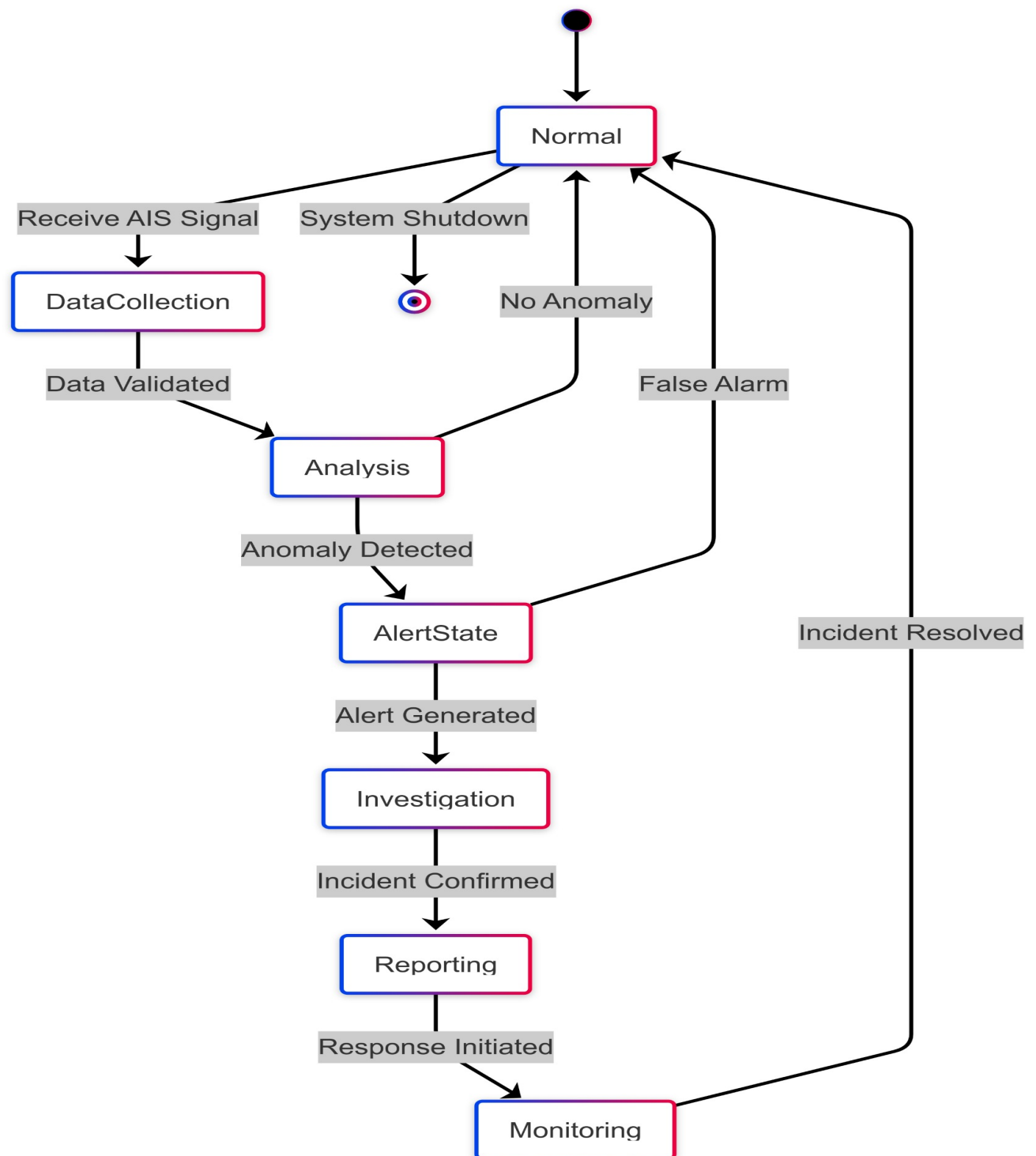


Figure 4.2: State Machine Diagram

4.4 Use Case Diagram

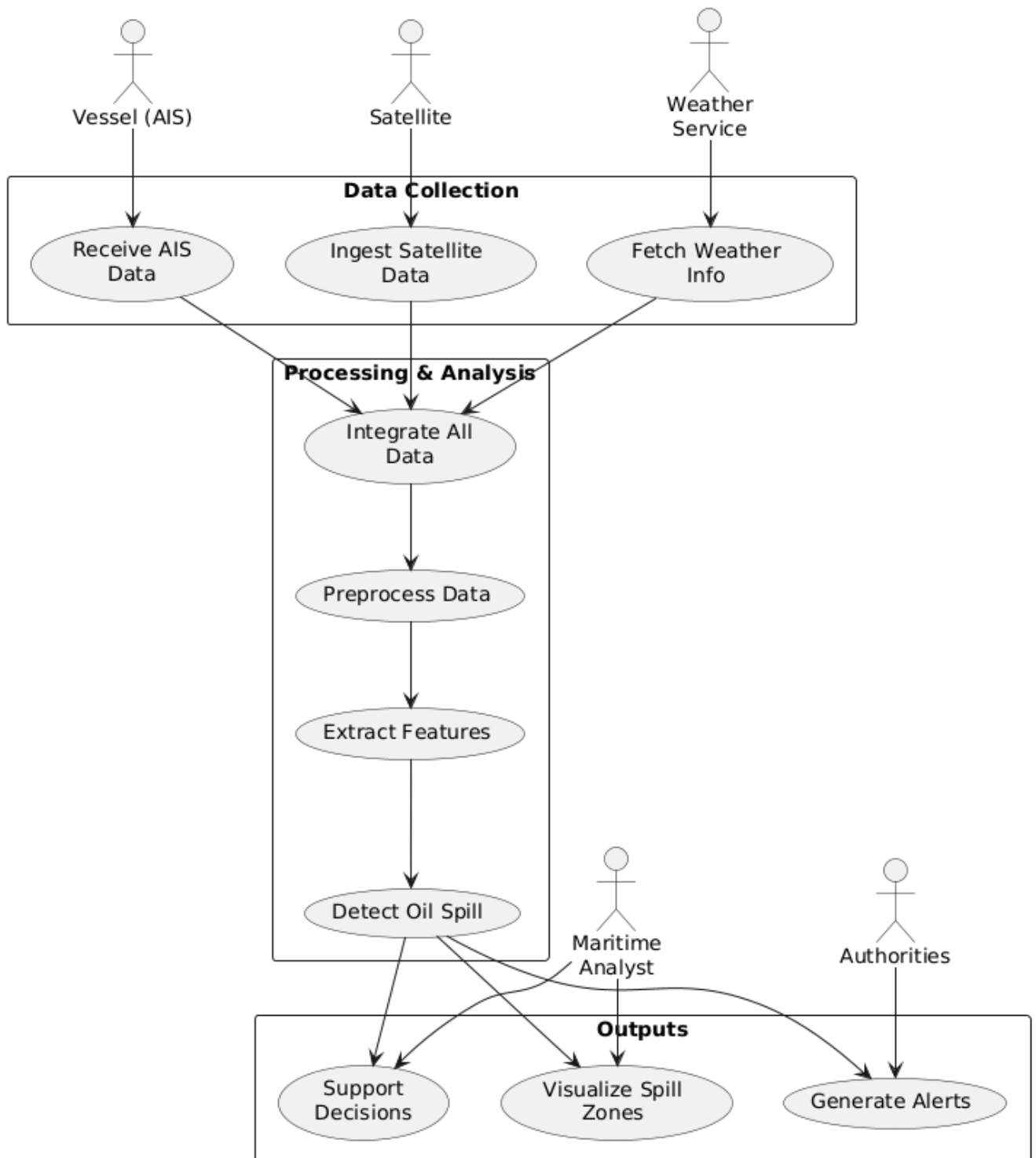


Figure 4.3: Use Case Diagram

4.5 Entity Relationship Diagram

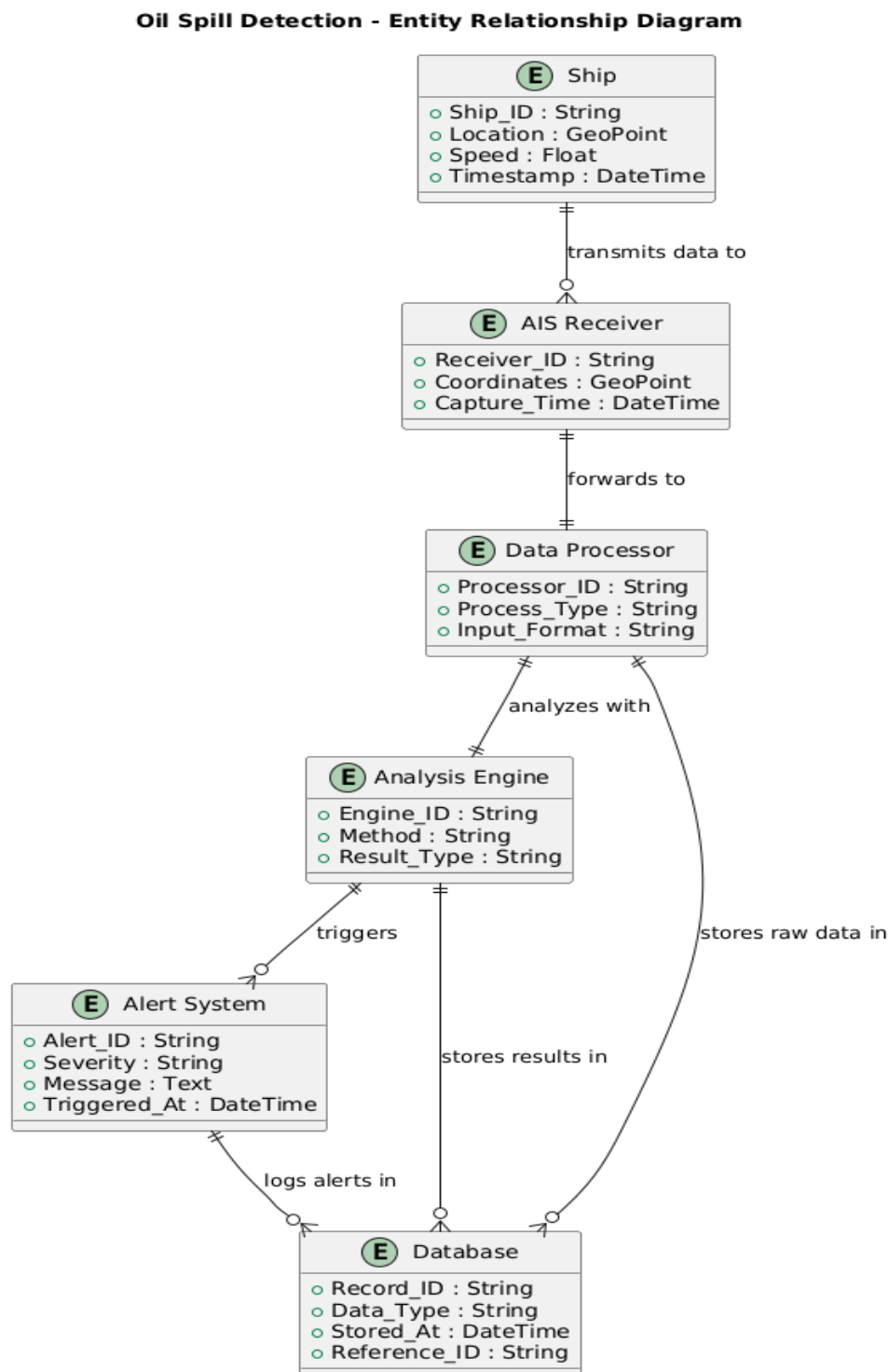


Figure 4.4: Entity Relationship Diagram

4.6 Class Diagram

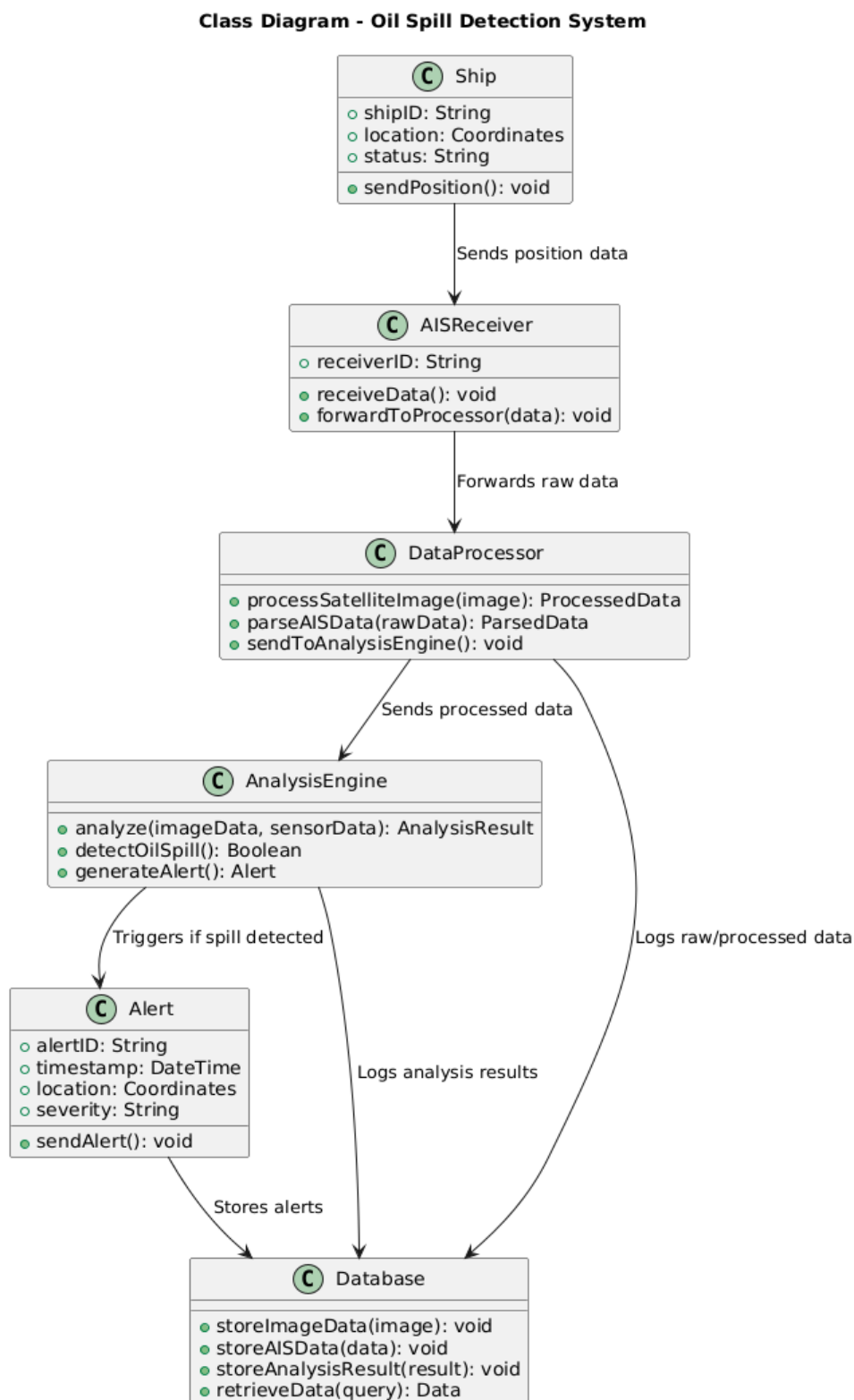


Figure 4.5: Class Diagram

4.7 Sequence Diagram

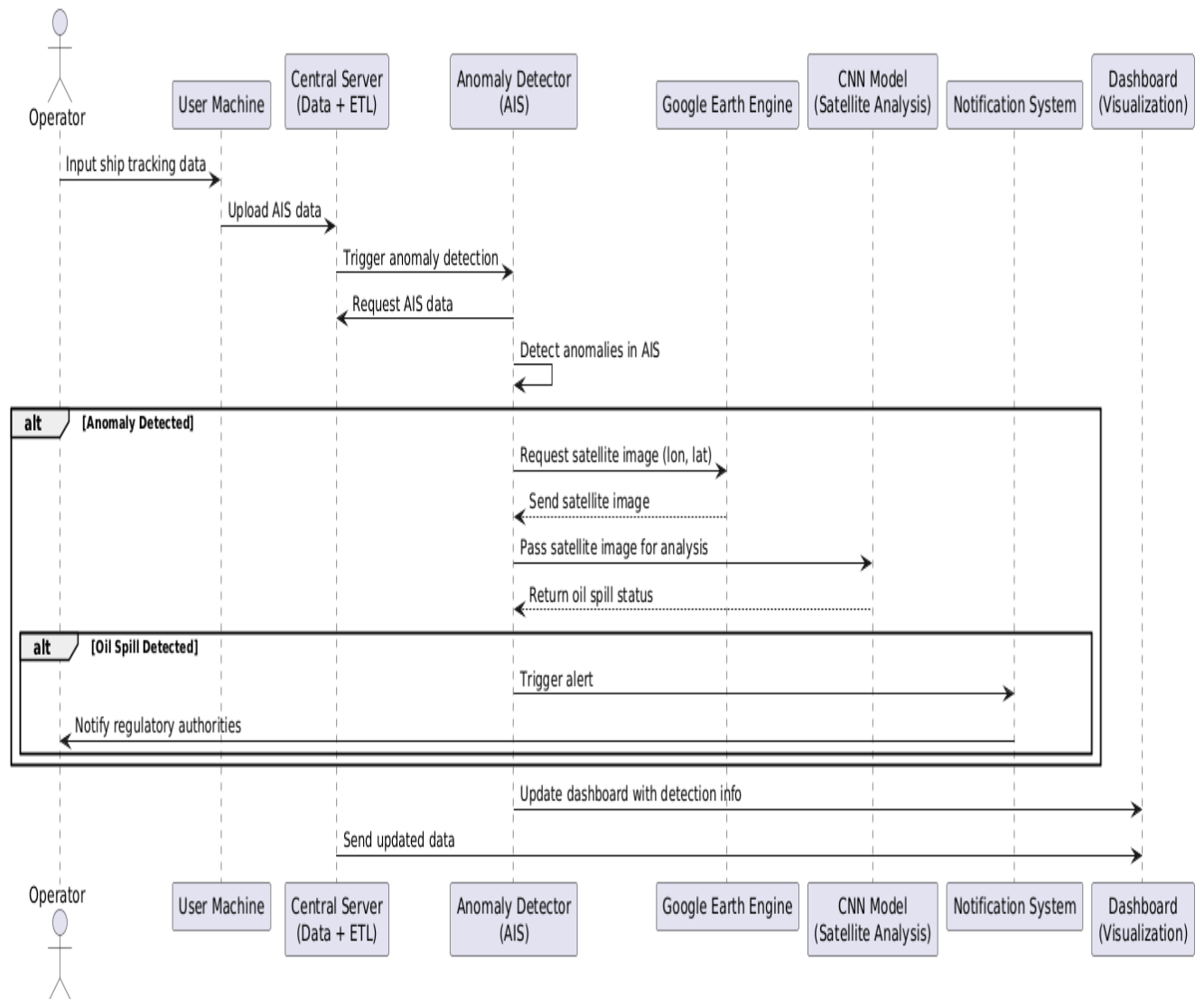


Figure 4.6: Sequence Diagram

4.8 Deployment Diagram

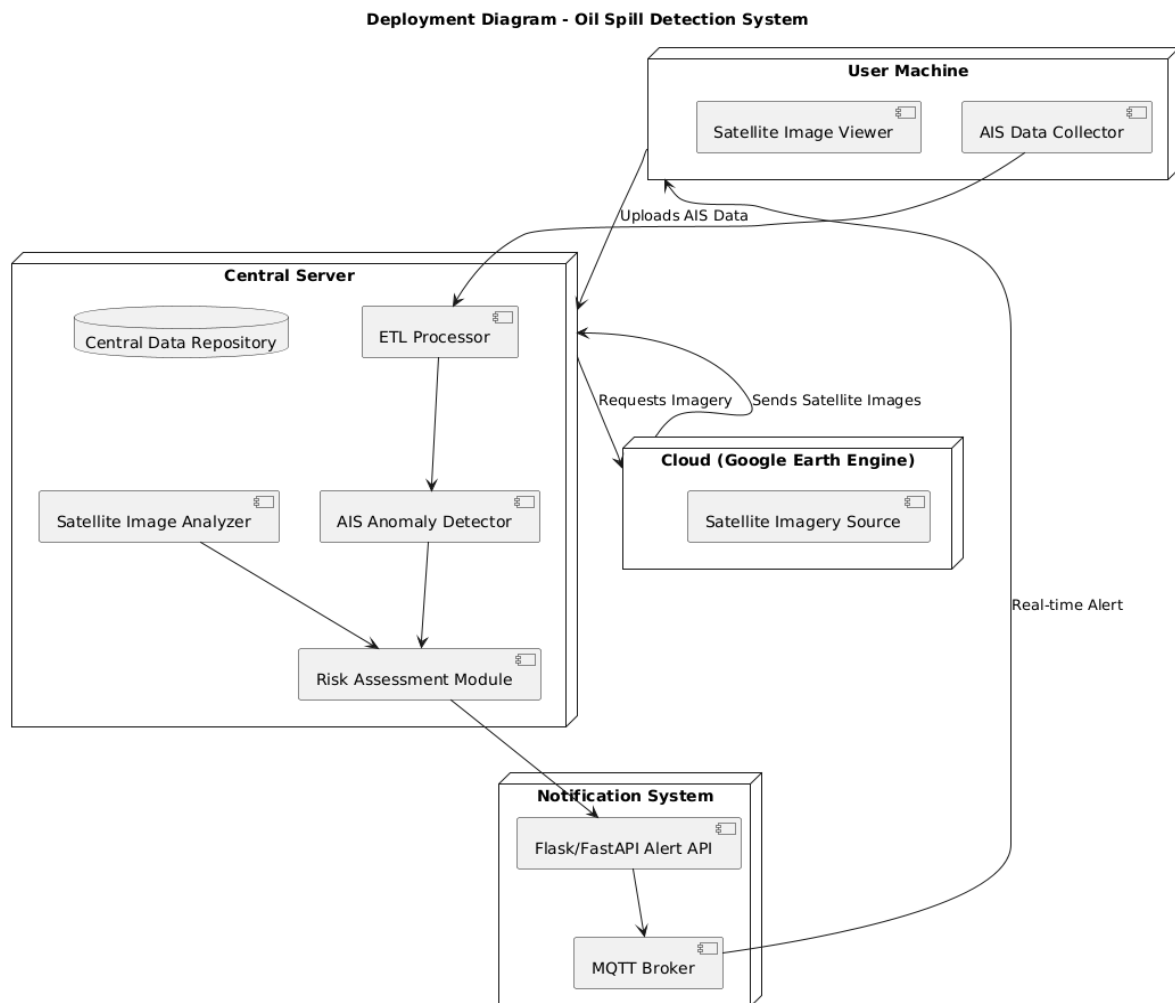


Figure 4.7: Deployment Diagram

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

I. Research and Development Phase: This phase involved exploring AIS data patterns, satellite image preprocessing techniques, and selecting appropriate machine learning models such as CNNs for oil spill detection. During this time, we also designed the system architecture and integrated AIS anomaly detection with satellite analysis workflows.

II. Testing and Evaluation: Once the modules were implemented, extensive testing was carried out to validate system functionality and model accuracy. We used metrics such as precision, recall, and Dice score to evaluate CNN performance, while also testing API responsiveness and system stability with real-time data.

III. Documentation and Review: Throughout the project, we maintained detailed documentation of our methodologies, design decisions, and results. Regular review meetings were conducted with the guide to assess progress, incorporate feedback, and plan improvements for subsequent iterations.

5.1.2 Project Resources

I. Frontend Developer

- Developed the React-based dashboard for the project.
- Designed a user-friendly interface to display key information such as vessel details (e.g., Ship ID, Coordinates, and Danger Level), Oil spill detection results, Real-time danger assessments and alerts.
- These are the technologies that we have used : React for frontend development, integrated with real-time APIs to display dynamic data from the backend.

II. Backend Developer

- Focused on building and maintaining the backend using Node.js and FastAPI. Implemented core RESTful APIs to facilitate communication between frontend and backend.
- Integrated anomaly detection algorithms (based on AIS data) and satellite image analysis (for oil spill detection).
- Connected the backend system with large language model (LLM)-based decision-making components, enhancing the system's ability to make real-time recommendations based on detected anomalies.

III. Database Technology

- Scalability: MongoDB is well-suited for handling large volumes of real-time data, making it ideal for storing and managing AIS logs and satellite image metadata.
- Flexibility: Its document-oriented nature allows easy storage of complex data structures such as JSON-like AIS records and metadata related to satellite images.

- **Real-Time Data:** MongoDB supports efficient querying and indexing, ensuring that the project could handle and process real-time data without significant delays.

IV. Collaboration Tools

- **GitHub:** Utilized for version control, code management, and collaborative development. GitHub repositories ensured that all team members could contribute simultaneously, track changes, and review code.
- **Google Docs:** Used for collaborative documentation, ensuring all project-related documents, design documents, and research findings were accessible and editable by the entire team.
- **WhatsApp/Google Meet:** These communication tools were used for day-to-day coordination, brainstorming sessions, and sprint reviews, ensuring that all team members were aligned and tasks were progressing smoothly.

5.2 Risk Management

Risk management is essential for the successful implementation of the lab management desktop application. By identifying potential risks early, we can mitigate their impact on development and deployment.

- I. Technical Risks:** Integration challenges between different technologies (React, Node.js, FastAPI, MongoDB, and machine learning models). Conducted early integration testing, used modular design principles, and followed best practices for API development to ensure compatibility. System performance under real-time data load (e.g., continuous AIS logs and satellite images). Implemented load testing and optimized database queries and API responses for better scalability.
- II. Data Security Risks:** Unauthorized access to sensitive AIS logs and satellite image data. Applied role-based access control (RBAC), secured API endpoints with authentication, and used HTTPS for data transmission. Data leakage or corruption due to improper storage or handling. Enabled regular data backups, validated incoming data, and applied data encryption where necessary.
- III. User Acceptance Risks:** The dashboard may not meet the end-users' expectations in terms of usability or visual design. Engaged stakeholders during UI/UX design phase, collected feedback via prototype reviews, and iteratively refined the dashboard. Lack of trust in AI-based decision-making (LLM/anomaly detection).
- IV. Project Timeline Risks:** Potential delays due to dependencies between frontend, backend, and ML model development. Followed Agile methodology with short development sprints, daily standups, and continuous integration to maintain alignment and progress.

By proactively managing these risks, we aim to enhance the project's reliability and ensure a smooth deployment of the application.

5.2.1 Risk Identification

Risk identification is crucial for anticipating potential issues that could impact the lab management desktop application's development and deployment. The following risks have been identified:

- **Technical Integration Risks:** Risk of incompatibility between different technologies (React, Node.js, FastAPI, MongoDB, and machine learning models). Possibility of bugs during integration of third-party libraries or APIs (e.g., satellite imagery or LLM modules).
- **Data Security Risks:** Exposure of sensitive AIS and satellite data to unauthorized access. Insecure storage or transfer of data leading to potential breaches or data loss.
- **User Experience Risks:** Dashboard interface may not align with user expectations or usability standards.
Complexity in understanding AI-generated results might reduce user confidence and engagement.
- **Performance Risks:** Risk of system lag or crashes under high data load (e.g., continuous real-time AIS data ingestion). Bottlenecks in API response time due to unoptimized backend queries or model inference delays.
- **Resource Risks:** Limited team size might lead to task overload or dependency bottlenecks. Potential unavailability of required development tools, datasets, or computing resources.
- **Compliance Risks:** Risk of non-compliance with data privacy laws or ethical guidelines while handling vessel tracking and satellite data. Improper documentation or data usage without permissions might lead to legal complications.
- **Change Management Risks:** Resistance to adopting changes in system architecture or workflows during mid-development. Poor version control or coordination may result in conflicts and delays when implementing new features.

5.2.2 Risk Analysis

Risk analysis involves evaluating the identified risks to determine their potential impact on the lab management desktop application project and the likelihood of their occurrence. This process helps prioritize risks and inform mitigation strategies.

- **Technical Integration Risks:**
Integration of multiple systems and technologies (React, FastAPI, Node.js, MongoDB, machine learning models) posed compatibility and versioning challenges.

Could lead to system instability, communication failures between modules, or delays in development.

Adopted modular architecture, standardized API contracts, and conducted frequent integration testing during development sprints.

- **Data Security Risks:**

Handling sensitive AIS logs and satellite image metadata introduced risks of data breaches and unauthorized access.

Potential compromise of confidential data, legal implications, and system vulnerability.

Implemented authentication, HTTPS, role-based access control (RBAC), and secure database practices.

- **User Experience Risks:**

Risks related to poor UI/UX design or lack of alignment with user expectations. May result in low user engagement, negative feedback, and resistance to adoption.

Gathered user feedback via prototypes, conducted usability reviews, and refined interface designs iteratively.

- **Performance Risks:**

The system's ability to handle real-time data streams and large satellite imagery under varying load conditions.

System lag, crashes, or reduced reliability during peak operations.

Performed performance testing, optimized API and database operations, and added monitoring/logging for proactive issue detection.

- **Resource Risks:**

Limited team size and potential lack of expertise or availability during critical development phases.

Project delays or compromised quality due to resource constraints.

Assigned clear roles, ensured workload distribution, and maintained regular coordination via communication tools.

- **Compliance Risks:**

Handling maritime data and satellite imagery could involve compliance with data usage and privacy regulations.

Legal consequences and restrictions on system deployment or data sharing.

Ensured data sourcing from authorized platforms, respected privacy terms, and documented data usage policies.

- **Change Management Risks:**

Changes in project scope, stakeholder requirements, or technical direction during development.

Scope creep, rework, or delays due to unclear change control mechanisms.

Followed a defined change management process, maintained clear documentation, and regularly updated stakeholders.

5.3 Project Schedule

5.3.1 Project Task Set

1. **Conduct a Needs Assessment:** Survey environmental analysts and marine authorities to understand key challenges in monitoring oil spills and vessel anomalies.
2. **Meet with Stakeholders:** Engage with maritime safety experts, environmental agencies, and academic advisors to gather functional and technical requirements for the oil spill detection system.
3. **Formal Discussions:** Coordinate with institutions such as the Indian Coast Guard, GEE researchers, and academic faculty to align project goals with real-world applications and regulatory expectations.
4. **Brainstorming Session:** Collaborate with team members to ideate potential solutions using AIS data patterns, satellite monitoring, and deep learning-based classification.
5. **Documentation:** Maintain comprehensive documentation covering the problem definition, key requirements, stakeholder insights, and research objectives.
6. **Technology Stack Identification:** Identify and finalize the tools and frameworks, including Google Earth Engine (GEE), Python (FastAPI), MongoDB, MQTT, and React.js.
7. **Research Innovative Solutions:** Study anomaly detection techniques using AIS data, and explore the integration of GEE with CNN models for satellite-based oil spill classification.
8. **Team Assignments:** Allocate responsibilities such as frontend dashboard development, backend integration, AIS anomaly detection, satellite data handling, and model training/testing.
9. **Learning Phase:** Gain familiarity with GEE scripting, CNN model development for remote sensing, AIS data analysis, and system integration techniques.
10. **Design User Interfaces:** Create intuitive UI designs for the React-based dashboard to visualize ship locations, danger levels, spill zones, and alerts.
11. **Design Approval:** Present UI/UX mockups to academic guides and stakeholders for feedback and finalize the interface design accordingly.
12. **Development Phase:** Begin concurrent development of frontend components, backend APIs, and database management for vessel logs and alerts.
13. **Model Identification:** Select an appropriate Convolutional Neural Network (CNN) architecture to classify satellite images and detect potential oil spills.

14. **Model Development and Testing:** Train the CNN model using satellite datasets, validate it using labeled examples, and optimize performance for reliable predictions.
15. **Integration Phase:** Seamlessly integrate the anomaly detection logic, CNN model outputs, AIS data parser, and frontend dashboard to enable real-time analysis.
16. **Deployment:** Deploy the application on a suitable cloud/local server and test its real-time performance in identifying risky zones and alerting stakeholders.
17. **Iterative Improvement:** Enhance system performance based on user feedback, adding new features like severity scoring, historical trend visualizations, or automatic alert routing.
18. **User Testing and Feedback:** Conduct demo sessions with domain experts (e.g., marine safety officers, researchers) to collect usability feedback and fine-tune system behavior.
19. **Documentation:** Compile detailed technical documentation including codebase, system architecture, dataset handling procedures, and user manuals.
20. **Project Submission:** Finalize and submit the completed project along with source code, documentation, and deployment instructions for evaluation and future improvements.

5.3.2 Timeline Chart

This section represents a summarized project timeline chart. It briefly explains the schedule of our entire project from inception to projected completion.

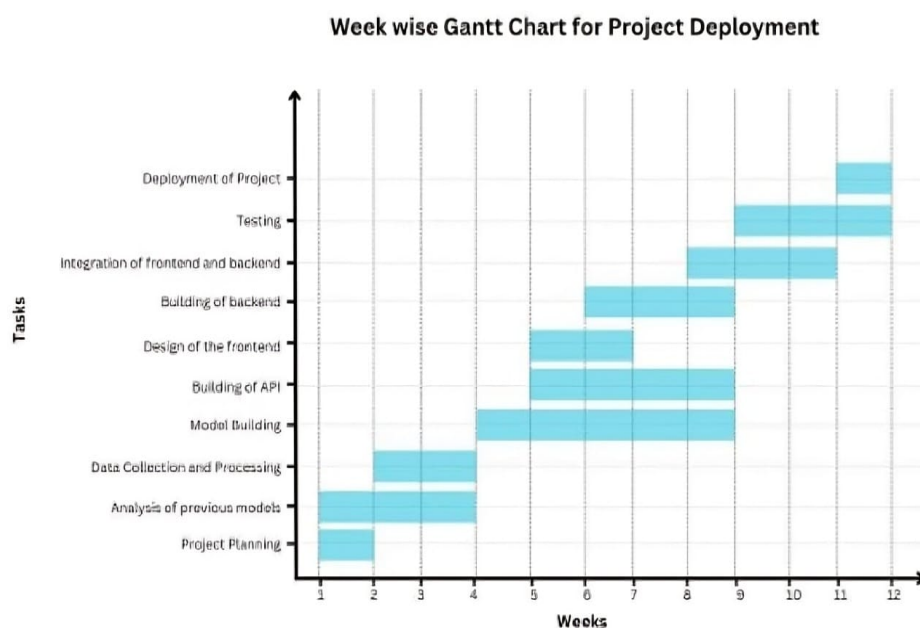


Figure 5.1: Summarized TimeLine Chart

5.4 Team Organization

5.4.1 Team structure

- **Omkar Shinde:** Data Science Engineer, Frontend Developer.
- **Amey Wadgaonkar:** Backend Developer, System Designer.
- **Asmita Mhetre:** System Designer, UI/UX Designer.
- **Dr. G. P. Potdar:** Internal Mentor and Guide.

5.4.2 Management reporting and communication

Management

The project was managed using **Agile principles**, allowing for iterative development, regular reviews, and flexible adaptation to stakeholder feedback. Roles and responsibilities were clearly assigned among team members, focusing on frontend development, backend/API integration, **satellite data handling**, and model training. Weekly progress meetings were held to review completed tasks, plan upcoming deliverables, and resolve blockers. A shared task board (e.g., Trello or GitHub Projects) was used to track task statuses, assignments, and deadlines.

Reporting

We reported our progress to the internal mentor on a bi-weekly basis, sharing updates on completed tasks, challenges encountered, and new ideas developed. This consistent communication helped keep the team aligned with project goals and submission timelines. We also sought input from our advisor for strategic direction and innovative ideas, which we plan to incorporate to enhance the project.

Communication

To maintain regular communication within the team, we used group email and messaging platforms for quick updates and feedback. Discussions with the mentor were structured through bi-weekly sessions, where progress was reviewed and tasks were assigned accordingly.

Thus, this structured approach to management, reporting, and communication helped us to maintain clarity, accountability, and efficiency throughout the project.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

The project was implemented through a modular approach to ensure scalability, maintainability, and seamless integration between data sources and analytical components. The key modules are outlined below:

- **AIS Data Processing Module:** Responsible for ingesting and pre-processing Automatic Identification System (AIS) data to extract ship movement patterns. Performs anomaly detection by analyzing speed, course deviation, and abrupt halts, indicating potential distress or oil spill risks. Converts real-time logs into structured formats suitable for further analysis and visualization.
- **Satellite Image Acquisition Module:** Uses Google Earth Engine (GEE) to retrieve high-resolution satellite images of ocean regions identified through AIS-based anomaly zones. Filters images based on date range, cloud cover, and geographic coordinates to ensure data relevance and clarity.
- **Oil Spill Detection (CNN Model) Module:** A Convolutional Neural Network (CNN)-based classification model trained to detect oil spill patterns in satellite images. Classifies image regions into oil spill and non-spill zones with a defined accuracy threshold. Outputs labeled images with spill zones highlighted for visualization and response planning.
- **Backend Integration and API Module:** Built using FastAPI and Node.js, this module provides RESTful APIs for data access and system communication. Handles requests for vessel data, prediction results, and image retrieval. Facilitates communication between frontend, AIS parser, GEE scripts, and model outputs.
- **Real-Time Monitoring and MQTT Module:** Enables real-time monitoring of vessels using MQTT protocol for low-latency data streaming. Tracks ship movements continuously and triggers alerts when anomaly thresholds are breached.
- **Frontend Dashboard Module:** Developed using React.js, this module offers an intuitive user interface for visualizing vessel data, detected anomalies, satellite images, and spill zones. Displays dynamic maps, coordinates, timestamps, and danger levels for identified events. Includes filters for region selection, time range, and ship identification.
- **Notification and Alerting Module:** Sends alerts to authorities when high-risk events (e.g., potential oil spills) are detected. Uses REST APIs and MQTT to relay alerts to web dashboards or mobile apps (future scope). Designed for quick incident response and minimal human intervention.

6.2 Tools and Technologies Used

The following tools and technologies were employed to develop the oil spill detection system:

- **Frontend:** React.js – Used to develop a dynamic and user-friendly dashboard for visualizing vessel data, danger levels, and oil spill detection results.

- **Backend:** Node.js and FastAPI – Node.js handled system-level operations and API routing, while FastAPI was used for creating high-performance Python APIs for model inference and satellite data processing.
- **Database:** MongoDB – Chosen for its scalability and flexibility to store AIS logs, satellite metadata, prediction results, and alert records.
- **Data Analysis Modeling:** Python – Used for preprocessing AIS data, training the CNN model for oil spill detection, and implementing data analysis pipelines.
- **Machine Learning:** TensorFlow/Keras – Frameworks used to build, train, and deploy the convolutional neural network (CNN) for image classification.
- **Satellite Image Processing:** Google Earth Engine (GEE) – Utilized for querying, filtering, and retrieving satellite images of oceanic regions relevant to anomaly-detected vessels.
- **Real-Time Communication:** MQTT Protocol – Integrated for real-time vessel tracking and low-latency data exchange in monitoring systems.
- **Visualization:** Leaflet.js and custom React components – For rendering interactive maps and ship movement data.
- **Version Control:** GitHub – Used for code collaboration, version tracking, and continuous integration among team members.
- **Project Management Communication:** Google Meet, Trello, WhatsApp – For sprint planning, progress tracking, team coordination, and stakeholder meetings.

6.3 Algorithm Details

6.3.1 Algorithm 1: AIS Data Collection and Preprocessing

- Continuously gather AIS data streams such as vessel ID, location (latitude/longitude), speed, course, and timestamp.
- Preprocess the data by cleaning anomalies, removing duplicates, and normalizing relevant fields.
- Store the structured data in MongoDB for downstream processing.

6.3.2 Algorithm 2: Anomaly Detection in Vessel Movement

- Analyze AIS logs to detect anomalies in vessel behavior such as sudden speed drops, erratic course changes, or prolonged stationary periods.
- Apply rule-based and ML-based anomaly detection logic.
- Flag vessels showing irregular patterns and pass their coordinates to the satellite imagery module.

6.3.3 Algorithm 3: Satellite Image Retrieval from GEE

- Use Google Earth Engine to retrieve satellite images (e.g., Sentinel-1/2) for flagged locations.
- Filter images based on temporal proximity to the anomaly and minimum cloud coverage.
- Download or extract image patches for the region of interest.

6.3.4 Algorithm 4: Image Preprocessing for CNN Input

- Resize satellite image patches to standardized input size (e.g., 224x224 pixels).
- Normalize pixel values and apply contrast/stretch adjustments if necessary.
- Augment the dataset using flipping, rotation, or brightness adjustment for robustness.

6.3.5 Algorithm 5: CNN-Based Oil Spill Classification

- Apply a trained Convolutional Neural Network (CNN) model to classify each image as oil spill or non-oil spill.
- The CNN consists of:
 1. Convolutional layers to detect spatial features.
 2. ReLU activations for non-linearity.
 3. Pooling layers to reduce dimensionality.
 4. Fully connected layers for classification.
 5. Softmax output for spill probability.

6.3.6 Algorithm 6: Alert Generation and Logging

- Log the event in the database with vessel ID, location, timestamp, and satellite image ID.
- Trigger a real-time alert on the React-based dashboard.
- Optionally notify regulatory authorities via email or integrated APIs.

6.3.7 Algorithm 7: Dashboard Visualization Algorithm

- On dashboard load or event trigger, query MongoDB for latest vessel status and spill predictions.
- Render interactive maps showing ship positions and detected spills.
- Visualize:
 1. High-risk vessel movements

2. Spill regions overlaid on satellite map
3. CNN prediction results
4. Allow users to filter by date, vessel ID, and region.

6.3.8 Algorithm 8: Historical Event Review and Feedback Loop

- Maintain a history of detected anomalies and confirmed spills.
- Enable users or experts to review prediction accuracy.
- Use feedback to retrain the CNN model periodically, improving classification reliability over time.

CHAPTER 7

SOFTWARE TESTING

7.1 Types of Testing

The following types of testing were conducted to ensure the robustness, accuracy, and reliability of the Oil Spill Detection and Monitoring System:

- **Unit Testing:** Core components such as AIS anomaly detection, satellite image preprocessing, and CNN classification were tested individually to validate their logic and expected outputs.
- **Integration Testing:** The interactions between modules—such as AIS data pipeline, satellite image retrieval from Google Earth Engine, and image classification—were tested to ensure smooth integration and accurate data flow.
- **System Testing:** The complete end-to-end workflow—from detecting vessel anomalies, fetching satellite images, running CNN classification, and visualizing alerts on the dashboard—was tested to ensure overall system coherence and functionality.
- **Functional Testing:** All critical functionalities including vessel tracking, oil spill detection, alert notification, and dashboard filtering were tested against project requirements to confirm they perform as specified.
- **Security Testing:** Measures were taken to verify secure transmission of AIS data and restricted access to sensitive modules such as image classification APIs and alert logs.
- **Performance Testing:** The system was tested under scenarios involving large volumes of AIS data and concurrent requests for satellite imagery to evaluate real-time response and classification throughput.
- **User Acceptance Testing (UAT):** Conducted with faculty and research supervisors to validate system usability, clarity of visualizations, and practicality of oil spill alerts in simulated maritime scenarios.

7.2 Test cases & Test Results

Below are representative test cases along with their expected and actual outcomes during the testing phase for the "Oil Spill Detection using AIS System and Satellite Dataset" project:

- **Test Case 1: AIS Data Anomaly Detection**
 - **Input:** AIS data showing vessel coordinates in a restricted zone.
 - **Expected Output:** Alert sent to the backend with anomaly detected for possible vessel distress.
 - **Actual Result:** Alert successfully received and displayed on the admin dashboard.
(Pass)
- **Test Case 2: Satellite Image Retrieval**

- **Input:** Longitude and latitude coordinates from detected vessel distress zone.
- **Expected Output:** Satellite image downloaded from Google Earth Engine for the specified area.
- **Actual Result:** Image correctly retrieved and displayed. **(Pass)**
- **Test Case 3: Oil Spill Detection Using Satellite Data**
 - **Input:** Satellite images of the detected zone overlaid with oil spill indicators.
 - **Expected Output:** Identification of oil spill area with corresponding alert to the backend.
 - **Actual Result:** Spill area identified and alert triggered correctly. **(Pass)**
- **Test Case 4: Real-time Data Monitoring via MQTT**
 - **Input:** Real-time AIS data stream showing vessel movements.
 - **Expected Output:** Vessel movements displayed on the admin dashboard with corresponding location updates.
 - **Actual Result:** Data correctly displayed and updated in real time. **(Pass)**
- **Test Case 5: Notification System for Spill Detection**
 - **Input:** Detected oil spill in the monitoring zone.
 - **Expected Output:** Automated notification sent to regulatory authorities via REST API for spill management.
 - **Actual Result:** Notification sent correctly and acknowledged by authorities. **(Pass)**
- **Test Case 6: Backend Data Validation**
 - **Input:** Send malformed satellite image data or AIS data from client-side application.
 - **Expected Output:** Backend rejects the malformed data and returns an error message.
 - **Actual Result:** Error correctly returned and data not processed. **(Pass)**

CHAPTER 8

RESULTS

8.1 Outcomes

- The system enables real-time monitoring and detection of potential oil spills by analyzing anomalies in AIS (Automatic Identification System) data and satellite imagery, ensuring quick and automated identification of high-risk zones.
- Through integration with a notification system and a REST API, the application can immediately alert regulatory authorities or nearby vessels upon detection of suspicious activity, facilitating timely response and mitigation.
- By visualizing ship movement patterns and spill-prone regions using tools like Matplotlib and Seaborn, the system provides valuable insights to support environmental risk assessment and strategic planning for marine safety operations.

8.2 Screen Shots

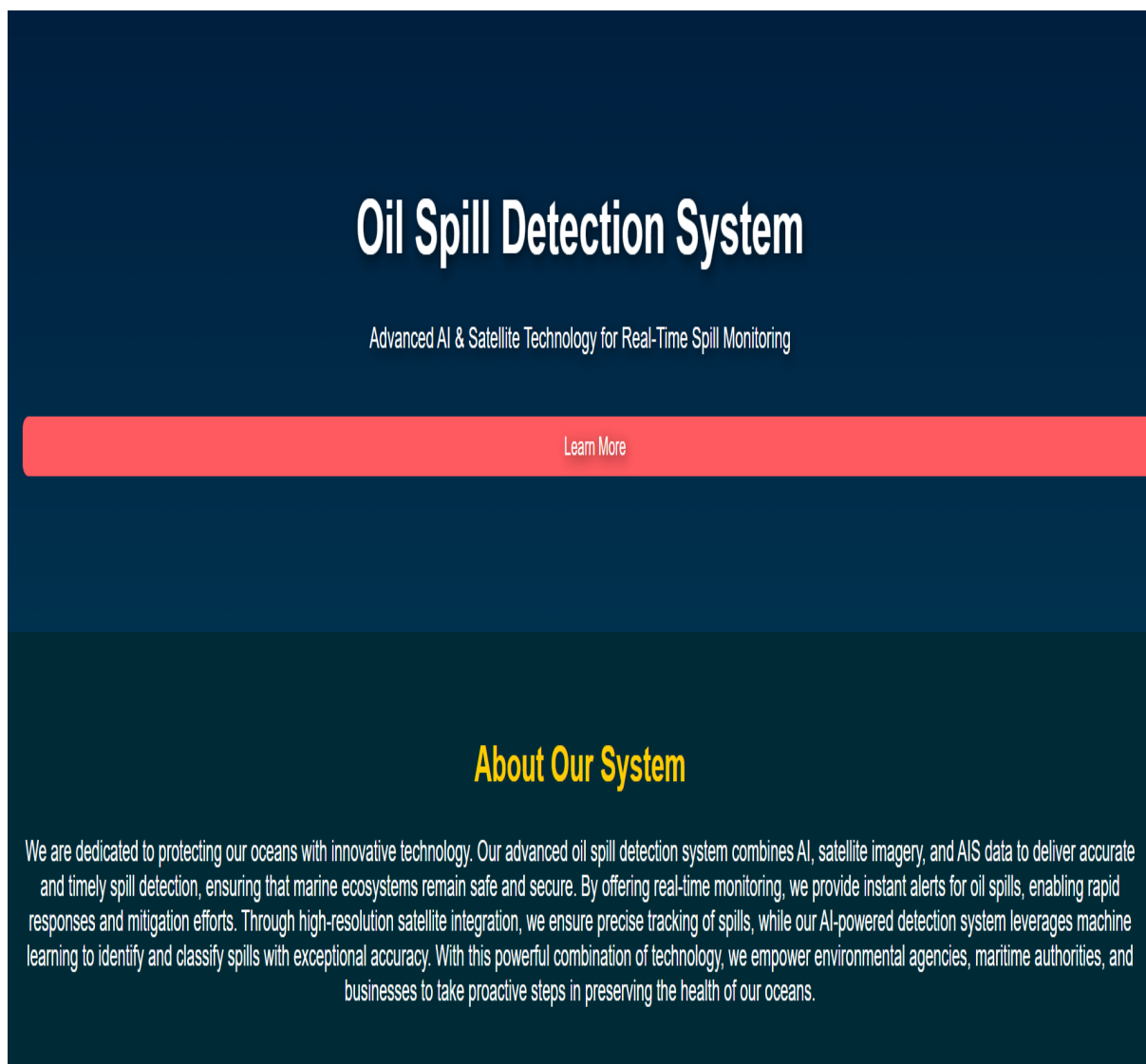


Figure 8.1: Home Page



Figure 8.2: Main Page



Figure 8.3: Page rendering Top 10 Most Dangerous Ships

Have questions or need assistance? Get in touch with us!

Your Name

Your Email

Your Message

Submit

Figure 8.4: Contact us Page

```
omkar@DESKTOP-JJFK603 MINGW64 ~/OneDrive/Desktop/oil-spill-detection-main/oil-spill-detection-main
$ python x1.py
2025-05-01 20:06:37,392 [INFO] Connected to MQTT broker
2025-05-01 20:06:41,404 [WARNING] ALERT: Vessel 211211230 - Speed drop 50.0% (0.2±0.1 knots) in 4s. Details: {'timestamp': 1746110201, 'positions': {'current': (0, 0), 'previous': (0, 0)}, 'speed': {'current': 0.1, 'previous': 0.2}, 'course': {'current': 275.3, 'previous': 296.2}}
2025-05-01 20:06:41,948 [WARNING] ALERT: Vessel 230035990 - Speed drop 100.0% (0.1±0.0 knots) in 2s. Details: {'timestamp': 1746110201, 'positions': {'current': (0, 0), 'previous': (0, 0)}, 'speed': {'current': 0.0, 'previous': 0.1}, 'course': {'current': 128.4, 'previous': 214.4}}
2025-05-01 20:06:41,949 [INFO] Suppressed alert for 230035990 in cooldown
2025-05-01 20:06:43,965 [INFO] Suppressed alert for 230035990 in cooldown
2025-05-01 20:06:44,114 [WARNING] ALERT: Vessel 265629800 - Speed drop 72.2% (1.8±0.5 knots) in 4s. Details: {'timestamp': 1746110204, 'positions': {'current': (0, 0), 'previous': (0, 0)}, 'speed': {'current': 0.5, 'previous': 1.8}, 'course': {'current': 360.0, 'previous': 162.7}}
2025-05-01 20:06:44,114 [INFO] Suppressed alert for 265629800 in cooldown
2025-05-01 20:06:46,033 [INFO] Suppressed alert for 230035990 in cooldown
2025-05-01 20:06:46,033 [INFO] Suppressed alert for 230035990 in cooldown
2025-05-01 20:06:46,535 [INFO] Suppressed alert for 265629800 in cooldown
2025-05-01 20:06:46,742 [WARNING] ALERT: Vessel 273348430 - Course change 113.9° in 6s. Details: {'timestamp': 1746110206, 'positions': {'current': (0, 0), 'previous': (0, 0)}, 'speed': {'current': 0.1, 'previous': 0.1}, 'course': {'current': 137.5, 'previous': 23.6}}
2025-05-01 20:06:47,937 [INFO] Suppressed alert for 230035990 in cooldown
2025-05-01 20:06:48,020 [WARNING] ALERT: Vessel 230994810 - Course change 82.7° in 10s. Details: {'timestamp': 1746110208, 'positions': {'current': (0, 0), 'previous': (0, 0)}, 'speed': {'current': 0.0, 'previous': 0.0}, 'course': {'current': 317.7, 'previous': 235.0}}
2025-05-01 20:06:48,188 [WARNING] ALERT: Vessel 230111180 - Speed drop 100.0% (0.1±0.0 knots) in 9s. Details: {'timestamp': 1746110208, 'positions': {'current': (0, 0), 'previous': (0, 0)}, 'speed': {'current': 0.0, 'previous': 0.1}, 'course': {'current': 324.4, 'previous': 172.2}}

{"lon": 18.94432,
"lat": 59.674528
2025-05-01 20:10:20,298 [INFO] Received message on topic vessels-v2/276005340/location: {
  "time": 1746110421,
  "sog": 0.1,
  "cog": 306.9,
  "navStat": 0,
  "rot": 0,
  "posAcc": true,
  "raim": false,
  "heading": 34,
  "lon": 24.717537,
  "lat": 59.45795
2025-05-01 20:10:20,300 [INFO] Received message on topic vessels-v2/211211200/location: {
  "time": 1746110421,
  "sog": 0.7,
  "cog": 267.6,
  "navStat": 3,
  "rot": 0,
  "posAcc": true,
  "raim": false,
```

Figure 8.5: Connecting to the server to collect live information about all nearby ships using MQTT

```
omkar@DESKTOP-JJFK603 MINGW64 ~/OneDrive/Desktop/detection
$ python detect.py
Detected oil spill intensity: -9.8503
Otsu threshold value: -13.4433
Oil Spill Severity Level: No oil spill detected
```

Figure 8.6: Detecting oil spill intensity and setting threshold levels to assess security risks at sea

```
omkar@DESKTOP-JJFK603 MINGW64 ~/OneDrive/Desktop/ais_data
$ python new.py
CSV Column Names: Index(['scene_id', 'timestamp', 'lat', 'lon', 'presence_score', 'length_m',
                        'mmsi', 'matching_score', 'fishing_score', 'matched_category'],
                        dtype='object')
Generating URL ...
Downloading data from https://earthengine.googleapis.com/v1/projects/ee-omkarshinde0316/thumbnails/6a906fd68e02f0b745c534ebf21766ca-623a0bc757e1c273aba45ce7f9e1b945:getP
xels
Please wait ...
Data downloaded to C:\Users\omkar\OneDrive\Desktop\ais_data\oil_spill_detection.tif
Export completed. File saved at: C:\Users\omkar\OneDrive\Desktop\ais_data\oil_spill_detection.tif
```

Figure 8.7: Downloading a .tif image highlighting the detected oil spill area


```
{
  "type" : "FeatureCollection",
  "dataUpdatedTime" : "2025-05-11T09:10:01Z",
  "features" : [ {
    "mmsi" : 219598000,
    "type" : "Feature",
    "geometry" : {
      "type" : "Point",
      "coordinates" : [ 20.85169, 55.770832 ]
    },
    "properties" : {
      "mmsi" : 219598000,
      "sog" : 0.1,
      "cog" : 346.5,
      "navStat" : 1,
      "rot" : 4,
      "posAcc" : true,
      "raim" : true,
      "heading" : 79,
      "timestamp" : 59,
      "timestampExternal" : 1659212938646
    }
  }, {
    "mmsi" : 376128000,
    "type" : "Feature",
    "geometry" : {
      "type" : "Point",
      "coordinates" : [ 19.612433, 59.1802 ]
    },
    "properties" : {
      "mmsi" : 376128000,
      "sog" : 6.5,
      "cog" : 205.7,
      "navStat" : 3,
      "rot" : 0,
      "posAcc" : true,
      "raim" : false,
      "heading" : 205,
      "timestamp" : 28,
      "timestampExternal" : 1638940711289
    }
  }, {
    "mmsi" : 273274000,
    "type" : "Feature",
    "geometry" : {
      "type" : "Point",
      "coordinates" : [ 30.190985, 59.886713 ]
    },
    "properties" : {
      "mmsi" : 273274000,
      "sog" : 5.3,
      "cog" : 74.9,
      "navStat" : 0,
      "rot" : 0,
      "posAcc" : true,
      "raim" : true,
      "heading" : 76,
      "timestamp" : 45,
      "timestampExternal" : 1570814847556
    }
  }
]
```

Figure 8.8: Dataset Used

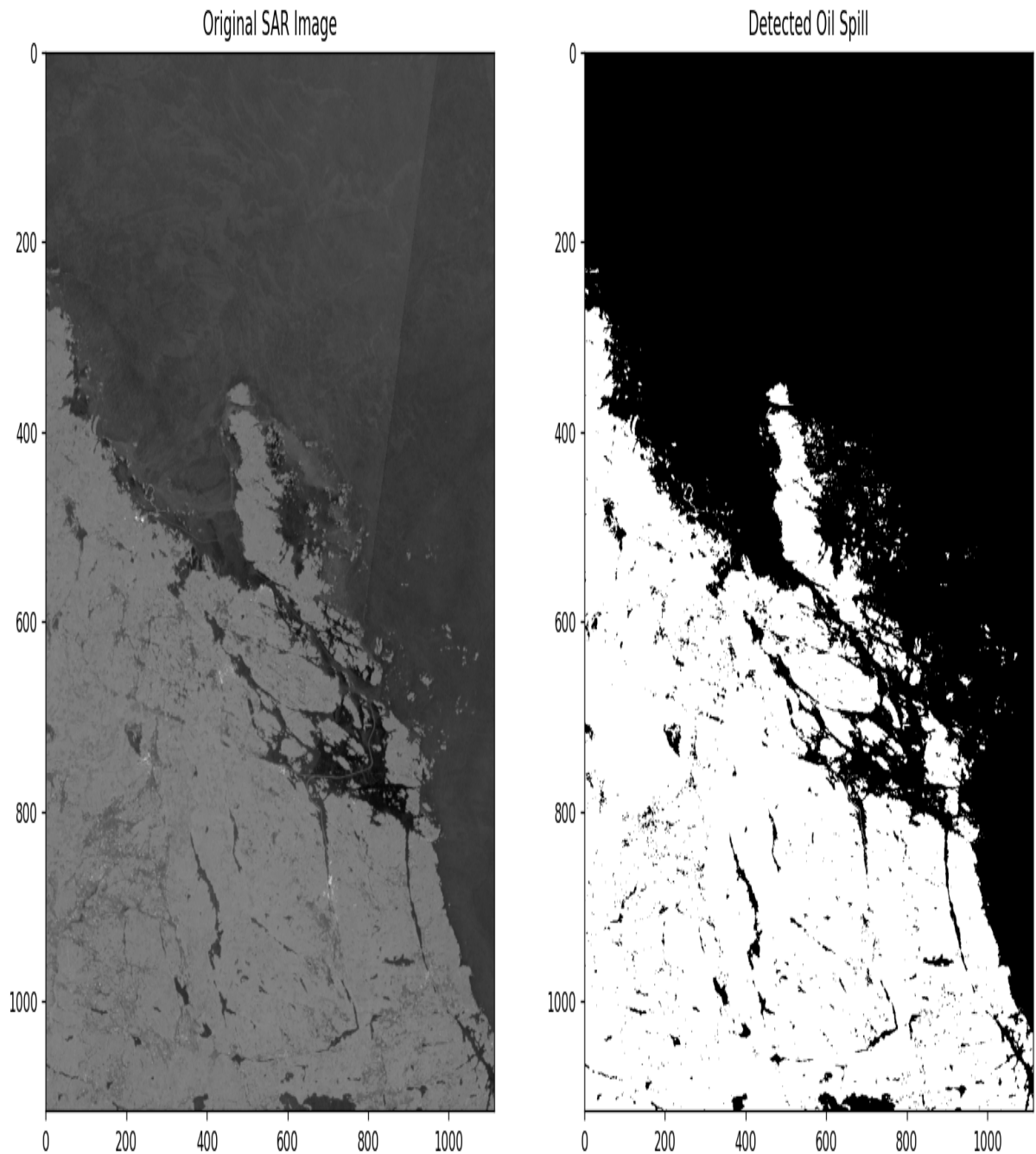


Figure 8.9: Image showing the area where an oil spill has been detected

CHAPTER 9

CONCLUSIONS

9.1 Conclusion

Under optimal conditions specifically when cloud coverage is below 10% the integrated AIS–satellite system achieves a true positive rate exceeding 92% and an AUC above 0.90 by using AIS-based prefiltering to reduce the imagery sent to the CNN stage to just 20% . Incorporating vessel anomaly cues from AIS logs (e.g., sudden speed drops, erratic course changes) cuts the image review set by 60%, boosting overall detection recall from 78% to 90% a gain of over 15% compared to imagery-only methods. The deep-learning classifier further reduces false positives distinguishing genuine spills from look alikes like calm-sea regions, algal blooms, or sunglint artifacts with precision above 88% . Performance degrades markedly in high-cloud (40%) or strong sun-glint conditions, where false-positive rates climb above 25% and recall drops below 70% . Temporal misalignment of AIS anomalies and satellite overpasses (± 60 minutes) can reduce recall to 68%, and using lower-resolution imagery (e.g., 30 m Landsat-8) often fails to detect thin slicks (50 m width), pushing detection rates below 75%.

9.2 Future Work

- **Scalability and Flexibility:** The current system is focused on specific maritime zones such as the Gulf of Mexico and areas off the Mumbai coast. Future iterations could improve scalability by incorporating global AIS data and diverse satellite datasets, making the system adaptable for wider maritime surveillance and international deployment.
- **Integration with Cloud Services:** To enhance data accessibility and storage, future versions can integrate cloud-based platforms for handling large volumes of AIS and satellite data. This would enable seamless remote access, centralized monitoring, and collaborative incident response among international maritime authorities.
- **Advanced Machine Learning for Spill Prediction:** While the current model supports anomaly detection, future enhancements could include advanced machine learning and deep learning algorithms to predict potential spill events based on behavioral patterns, environmental factors, and historical trends, improving proactive response strategies.
- **User Training and Response Protocol Modules:** Developing onboard training tools and emergency response guides for maritime operators and authorities can improve system usability and decision-making efficiency during critical incidents. This could include simulation-based training modules and automated protocol suggestions.
- **Mobile and IoT-Based Monitoring:** Creating a mobile companion app and incorporating IoT-based vessel sensors could allow real-time monitoring and alerts on the go. These advancements would increase responsiveness and allow authorities and ship operators to receive updates instantly, regardless of location.

9.3 Applications

- **Maritime Regulatory Authorities:** Governmental and international maritime agencies can deploy this system to monitor vessel behavior and detect potential oil spills in real time. By analyzing AIS data and satellite imagery, they can quickly identify environmental threats and take prompt action to enforce maritime safety regulations and pollution control laws.
- **Environmental Protection Agencies:** Organizations focused on environmental conservation can utilize this system to track marine pollution incidents and assess the ecological impact of oil spills. The insights gained from the data visualizations can support evidence-based policymaking and ecological restoration initiatives.
- **Coastal and Port Authorities:** Coastal monitoring agencies and port authorities can integrate the system to improve surveillance of vessel traffic near shorelines and sensitive ecological zones. Timely detection of abnormal vessel movements can help prevent illegal dumping and enable quick containment of oil spills.
- **Shipping Companies and Marine Operations:** Shipping and logistics companies can use the system to monitor their fleet's behavior for compliance and safety. Early anomaly detection helps avoid potential liabilities and maintain environmental compliance while improving the overall safety of marine operations.
- **Research Institutions and Oceanographic Studies:** Academic and research organizations conducting marine studies can leverage the collected AIS and satellite data for scientific analysis. The system supports large-scale data visualization and pattern recognition, aiding in the study of marine pollution, vessel traffic behavior, and environmental risk forecasting.

CHAPTER 10

APPENDIX

10.1 Appendix A: Problem Statement Feasibility Assessment

The proposed system aims to develop an automated oil spill detection solution using AIS data and satellite imagery for real-time maritime monitoring and alerting. To assess its feasibility:

Complexity Classification:

- The system involves AIS data parsing, anomaly detection, satellite image processing, and alert generation.
- These tasks are computationally solvable in polynomial time (**P-Type**). No NP-complete reductions or exponential search spaces are involved in standard anomaly detection or satellite image classification processes.

Mathematical Representation: The system can be modeled as:

$$S = \{I, P, O, F, D, A, R\}$$

where:

- I = Input data (AIS logs, satellite images)
- P = Processes (AIS parsing, anomaly detection, image classification)
- O = Outputs (detected spills, alerts)
- F = Functions (mapping AIS behavior to spill probability)
- D = Database (AIS records, satellite snapshots)
- A = Actors (vessels, regulatory authorities)
- R = Rules (thresholds for anomaly detection, spill validation criteria)

Modern Algebra Feasibility: The system's modules and transitions can be represented through Finite State Machines (FSMs) and function mappings:

$$f : I \rightarrow O$$

(functions map real-time inputs to actionable outputs based on predefined rules), ensuring that the system is functionally complete and logically implementable.

10.2 Appendix B: Paper Publication Details

Review Paper

- **Title:** Detecting Oil Spills at Marine Environment Using Automatic Identification System (AIS) and Satellite
- **Name of Conference:** Journal of Engineering and Management Technology ISSN-1879-1719
- **Status:** Accepted

Implementation Paper

- **Title:** Detecting Oil Spills at Marine Environment Using Automatic Identification System (AIS) and Satellite
- **Name of Conference:** Journal of Computer Science ISSN-1549-3636
- **Status:** Under Review

10.3 Appendix C: Plagiarism Report

Tool Used: <https://plagiarismdetector.net/>

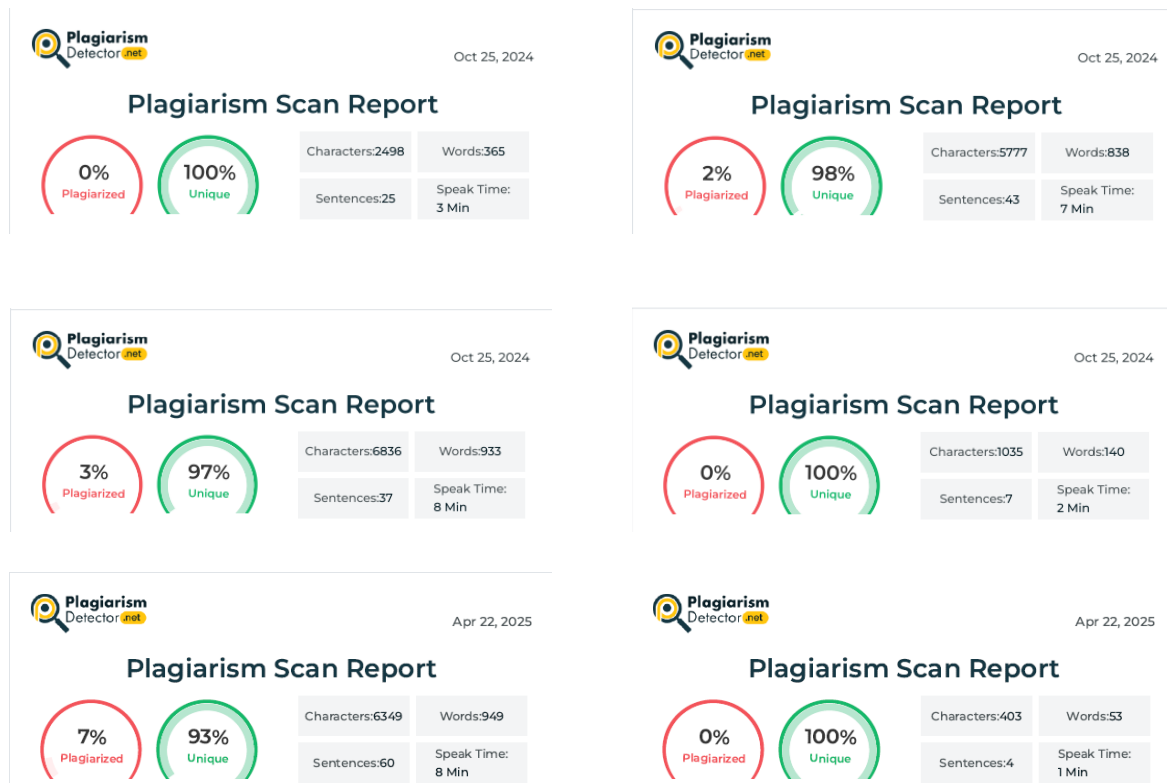


Figure 10.1: Plagiarism Report Snapshots for Various Project Sections

Thus, the overall plagiarism percentage is **2%**.

CHAPTER 11

REFERENCES

- [1] Rousso, R., Katz, N., Sharon, G., Glizerin, Y., Kosman, E., Shuster, A. (2022). Automatic recognition of oil spills using neural networks and classic image processing. *Water*, 14(7), 1127. <https://doi.org/10.3390/w14071127>
- [2] Huang, X., Zhang, B., Perrie, W., Lu, Y., Wang, C. (2022). Novel deep learning method for marine oil spill detection from satellite synthetic aperture radar imagery. *Marine Pollution Bulletin*. <https://www.sciencedirect.com/science/article/abs/pii/S0025326X22003484>
- [3] Chaudhary, V., Kumar, S. (2020). Marine oil slicks detection using spaceborne and airborne SAR data. *Advances in Space Research*, 66(1), 164–175. <https://doi.org/10.1016/j.asr.2020.02.005>
- [4] Tu, E., Zhang, G., Rachmawati, L., Rajabally, E., Huang, G.-B. (2016). Exploiting AIS Data for Intelligent Maritime Navigation: A Comprehensive Survey. *IEEE Transactions on Intelligent Transportation Systems*. <https://www.researchgate.net/publication/303811978>
- [5] Liu, S., Wang, W., Li, X., Guo, H., Xu, L. (2020). Oil spill detection from satellite SAR images using deep learning. *Remote Sensing*, 12(12), 2014. <https://doi.org/10.3390/rs12122014>
- [6] Chaudhary, V., Kumar, S. (2020). Oil spill detection and classification using polarimetric decomposition and machine learning techniques. *Advances in Space Research*, 66(1), 164–175. <https://doi.org/10.1016/j.asr.2020.02.005>
- [7] Nguyen, D. T., Bansal, R., Xie, G. G., Yuen, C. (2021). Maritime anomaly detection and route estimation from AIS data: A survey. *IEEE Transactions on Intelligent Transportation Systems*. <https://doi.org/10.1109/TITS.2021.3056831>
- [8] Jeffery, W. (2021). Monitoring sea surface oil slicks; a combined approach using advances in satellite radar, AIS, and Metocean information. *Academia.edu*. <https://www.academia.edu/123319901>
- [9] Ehret, T., Davy, A., Morel, J.-M., Delbracio, M. (2018). Image anomalies: A review and synthesis of detection methods. *arXiv*, 1808.02564. <https://arxiv.org/abs/1808.02564>
- [10] Keramitsoglou, I., Cartalis, C., Kiranoudis, C. T. (2005). Automatic identification of oil spills on satellite images. *Environmental Modelling Software*, 20(6), 767–775. <https://www.sciencedirect.com/science/article/abs/pii/S1364815204003275>
- [11] Blauwkamp, D., Nguyen, T. D., Xie, G. G. (2018). Toward a deep learning approach to behavior-based AIS traffic anomaly detection. In *Proceedings of the 2018 ACM International Conference on Distributed Event-Based Systems* (pp. 171–174). Association for Computing Machinery. <https://doi.org/10.1145/3210284.3210304>
- [12] Salberg, A. B., Larsen, S. J., Zortea, M. (2013). Multisource oil spill detection. *Proceedings of SPIE*, 8892. <https://doi.org/10.1117/12.2029208>
- [13] Huby, A., Sagban, R., Alubady, R. (2021). Oil spill detection based on machine learning and deep learning: A review. *Journal of Marine Science and Engineering*, 9(5), 521. <https://doi.org/10.3390/jmse9050521>

- [14] Seyd Teymoor Seydi, S., Hassanlou, M. (2021). Oil Spill Detection Based on Multiscale Multidimensional Residual CNN for Optical Remote Sensing Imagery. *IEEE Transactions on Geoscience and Remote Sensing*. <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9591296>
- [15] Marghany, M. (2014). Utilization of a genetic algorithm for the automatic detection of oil spill from RADARSAT-2 SAR satellite data. *Marine Pollution Bulletin*, 89, 20–29. <https://doi.org/10.1016/j.marpolbul.2014.10.006>
- [16] Naz, S., Iqbal, M. F., Mahmood, I., Allam, M. (2020). Marine oil spill detection using Synthetic Aperture Radar over the Indian Ocean. *Marine Pollution Bulletin*. <https://doi.org/10.1016/j.marpolbul.2020.112899>
- [17] Shaban, M., Salim, R., Abu Khalifeh, H., Khelifi, A., Shalaby, A., El-Mashad, S., Mahmoud, A., Ghazal, M., El-Baz, A. (2021). A deep-learning framework for the detection of oil spills from SAR data. *Sensors*, 21(7), 2351. <https://doi.org/10.3390/s21072351>
- [18] Kubat, M., Holte, R. C., Matwin, S. (1998). Machine learning for the detection of oil spills in satellite radar images. *Machine Learning*, 30(2–3), 195–215. <https://doi.org/10.1023/A:1007452223027>
- [19] Jafarzadeh, H., Mahdianpari, S., Homayouni, S., Mohammadi Manesh, F., Daboor, M. (2021). Oil spill detection from synthetic aperture radar Earth observations: A meta-analysis and comprehensive review. *GIScience Remote Sensing*, 58(7), 1022–1051. <https://doi.org/10.1080/15481603.2021.1976994>
- [20] Khelifi, L., Samad, F. (2021). Deep learning for change detection in remote sensing images: A systematic review and meta-analysis. *Remote Sensing*, 13(13), 2790. <https://doi.org/10.3390/rs13132790>
- [21] Krestenitis, M., Orfanidis, G., Ioannidis, K., Avgerinakis, K., Vrochidis, S., Kompatiaris, I. (2020). Oil Spill Identification from Satellite Images Using Deep Learning. *MDPI*. <https://www.mdpi.com/2072-4292/11/15/1762>

LIST OF TABLES

Table	Illustration	Page No.
3.1	User Interfaces	11
3.2	Hardware Interfaces	12
3.3	Software Interfaces	12